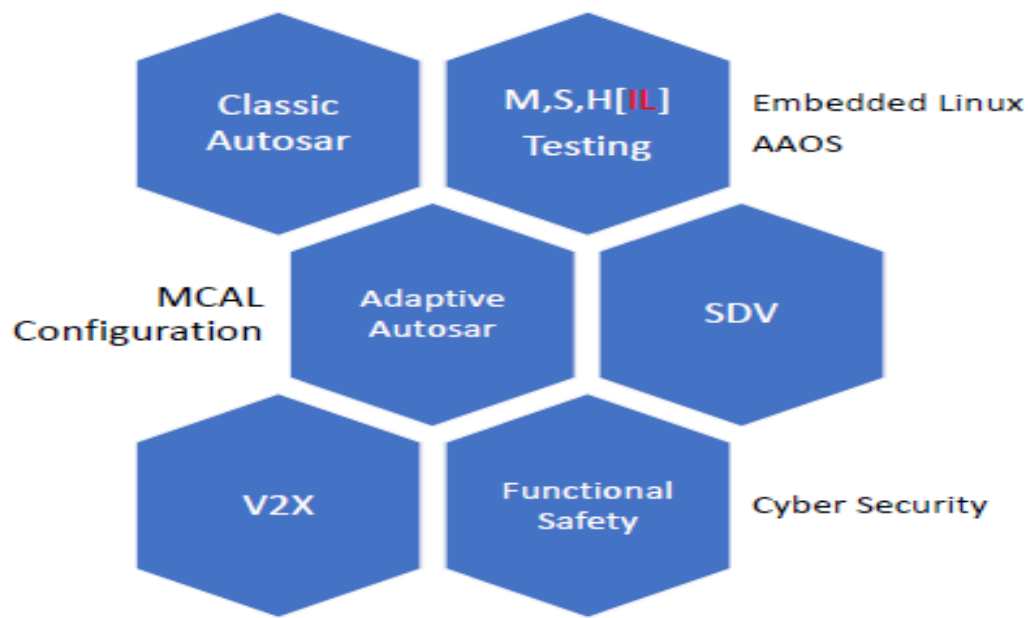




# ANCIT OPEN ECU Trainer KIT STM MANUAL

## Our Competency Cluster



## About ANCIT

ANCIT is a rapidly growing Automotive Software Engineering service provider with a team of over 150 professionals. We collaborate with renowned clients such as BOSCH, Daimler, NVIDIA, Infineon, ZF, TCS, HCL, Accenture, and Royal Enfield, delivering Consulting, Development, and Managed Services across the V Model in the automotive sector.

ANCIT provides tailored Proof of Concept solutions and product development, both offshore and onsite. We offer expert design guidance, performance optimization, and on-call support, along with workshops and customized training sessions for specific project needs.

Our mission is to provide top-tier consulting and tooling services to OEMs, Tier 1 suppliers, semiconductor companies, Fortune 500 firms, and innovative startups. Our expertise spans a wide range of services, from upskilling and consulting to managed services and process automation, all aimed at enhancing productivity.

Our expertise includes:

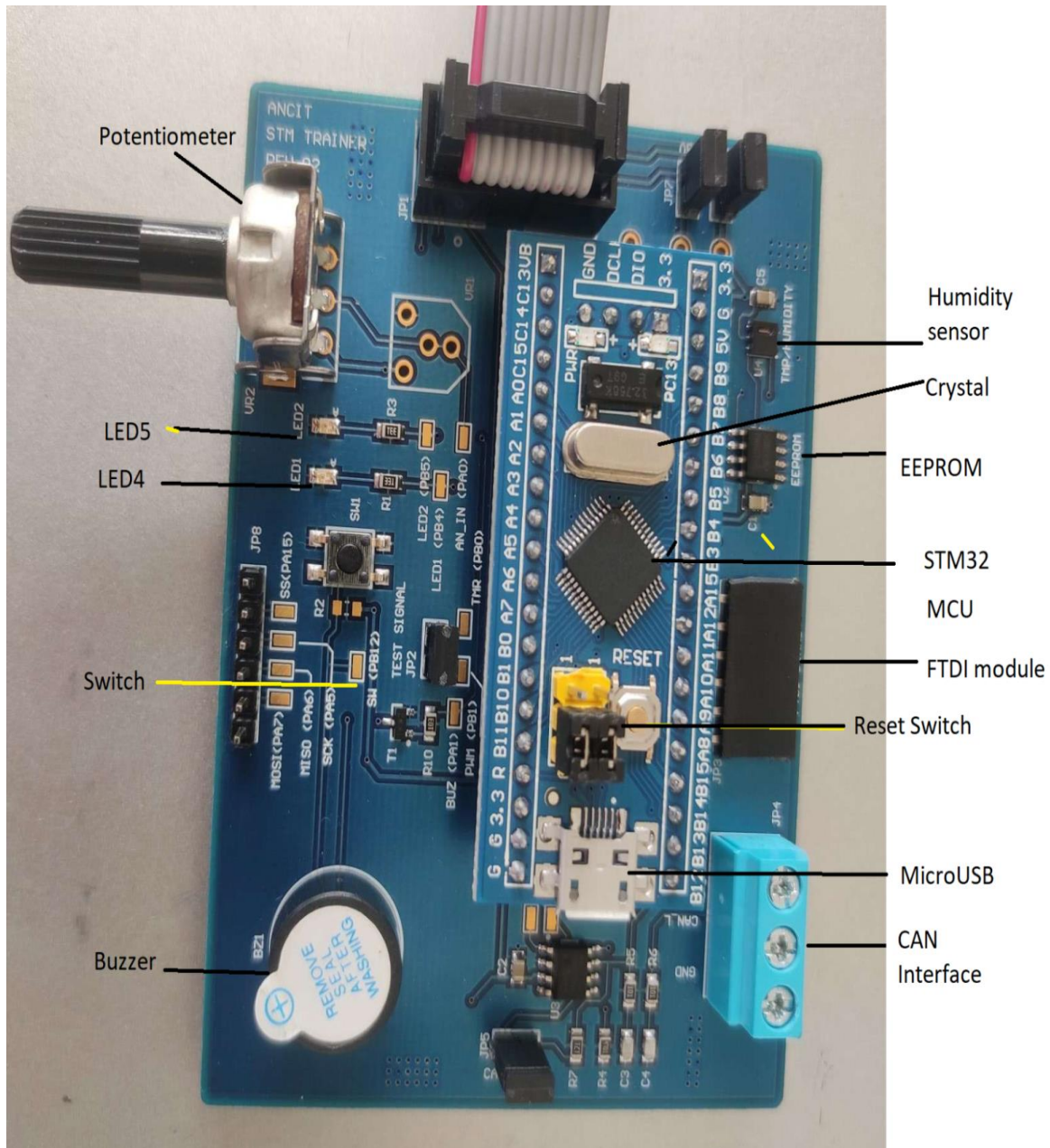
- VCU Development: Designing robust Vehicle Control Units to enhance system performance and reliability.
- AUTOSAR Services: As AUTOSAR consortium members and Elektrobit partners, we provide comprehensive architecture design, development, and integration services.
- Open ECU Solutions: Customized systems that seamlessly integrate with existing technology, ensuring flexibility and scalability.
- ANCIT STM Trainer Board: An affordable embedded systems trainer board featuring the STM32F103C8T6 controller, supporting various peripherals and ideal for training purposes.

With offices in Bangalore, Coimbatore, and Dindigul, and plans for expansion to Hyderabad and Pune, ANCIT also runs a subsidiary, ANCIT EduTech, focused on niche skill development in Automotive ECU Development and Testing. Our committed team supports clients throughout the entire product lifecycle, from R&D to post-production, ensuring comprehensive assistance and training tailored to specific project needs.

## Table of Contents

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>MANUAL FOR ANCIT BOARD .....</b>                        | <b>3</b>  |
| <b>SOFTWARE INSTALLATION .....</b>                         | <b>4</b>  |
| <b>PROGRAMS .....</b>                                      | <b>6</b>  |
| 1. Toggle an LED .....                                     | 6         |
| 2. Switch Operation .....                                  | 9         |
| 3. Breathing of LED's (PWM) .....                          | 11        |
| 4. ADC Operation .....                                     | 13        |
| 5. UART Working .....                                      | 15        |
| 6. Timer Interrupt .....                                   | 18        |
| 7. CAN TX and RX .....                                     | 21        |
| <b>BUSMASTER INSTALLATION .....</b>                        | <b>27</b> |
| <b>INNOMAKER DOWNLOAD PROCESS .....</b>                    | <b>31</b> |
| <b>DEMONSTRATE KITS CREATED USING ANCIT OPEN ECU .....</b> | <b>33</b> |
| <b>COURSES OFFERED BY ANCIT .....</b>                      | <b>35</b> |

# MANUAL FOR ANCIT BOARD



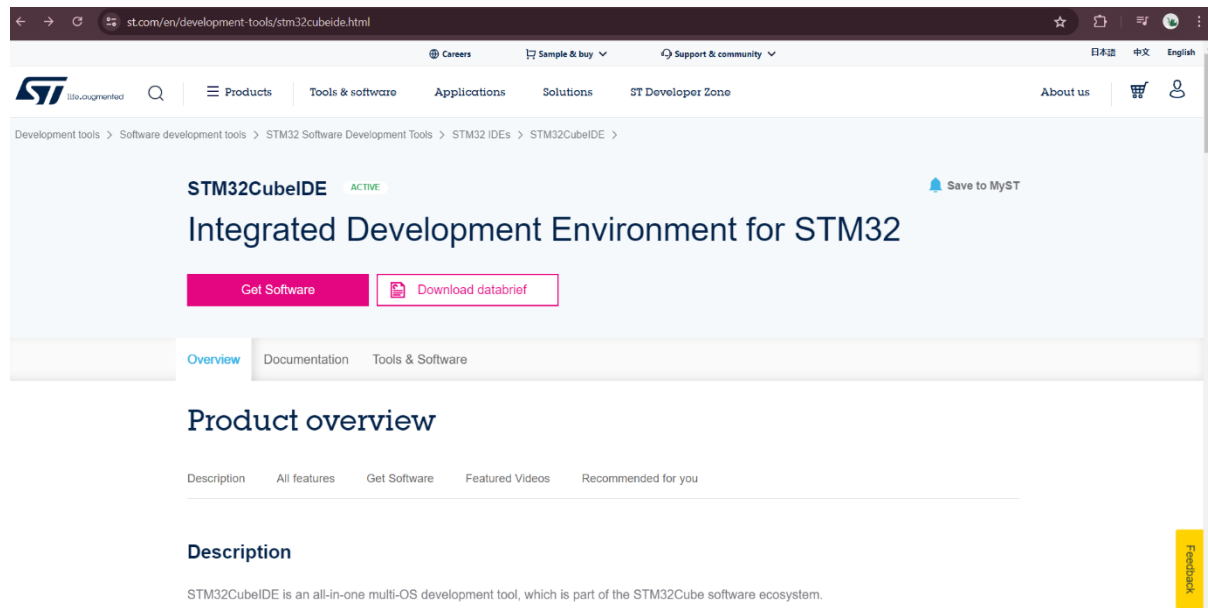
This is the Ancit Board. It is developed from the STM32F103C6T6 and the features that are provided by this board are shown above. ST link is used to flash the code.

# SOFTWARE INSTALLATION

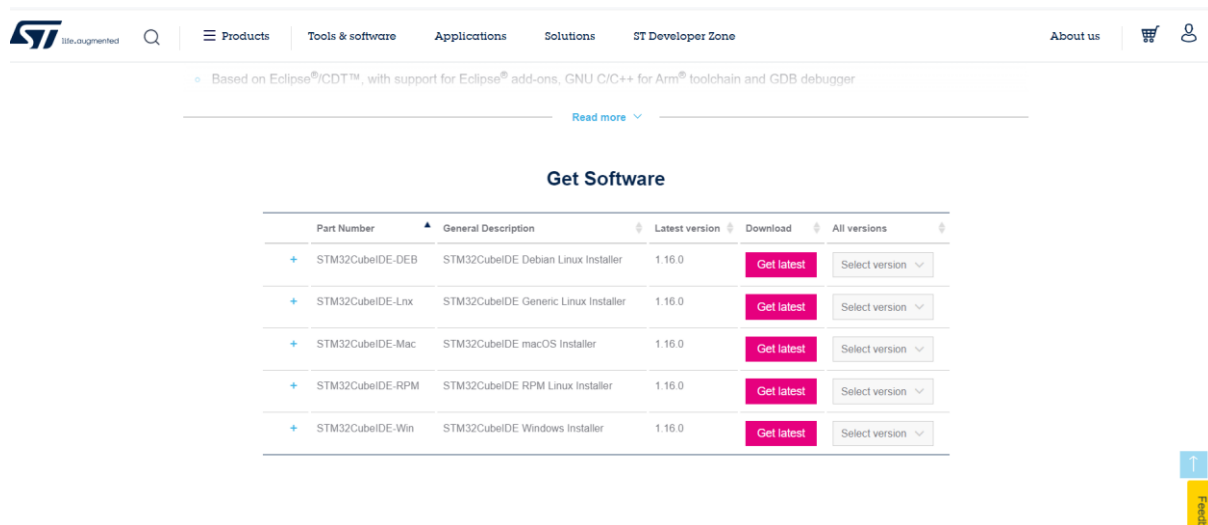
The software we use for the programming of STM board is STM32CubeIDE. For the installing, go to the below link or in google search type STM32CubeIDE and click on the first link.

<https://www.st.com/en/development-tools/stm32cubeide.html>

After clicking the link, on the right-side top u will have the login option click on that and create your account, then after click on get software.



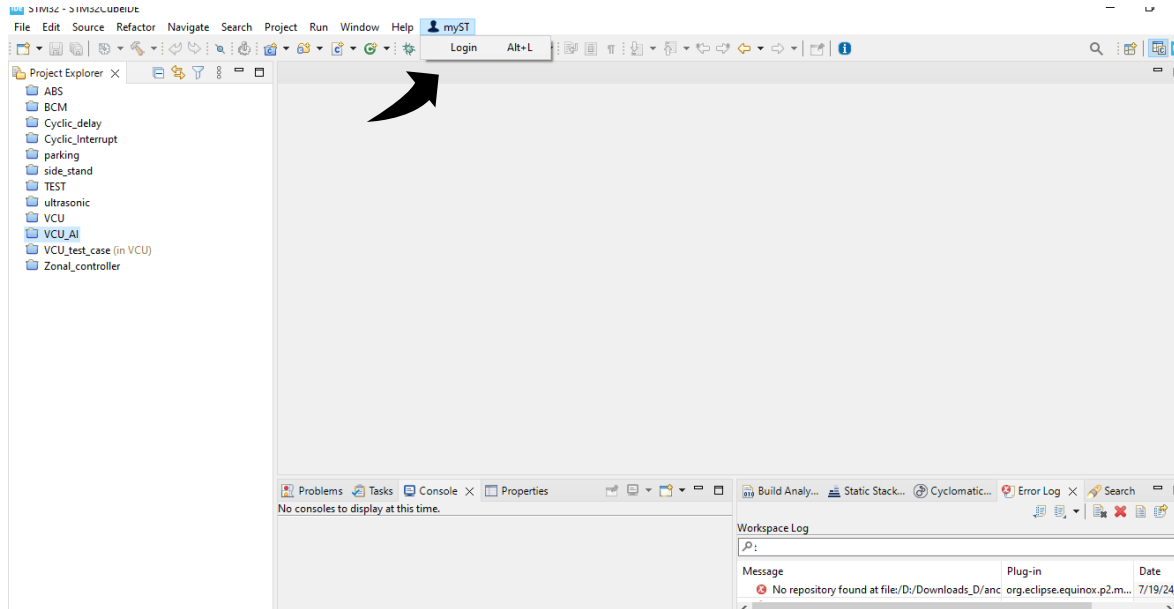
Now you can see download options. From that you can download the software.



After downloading, install the software into your system then open STM32CubeIDE.



After opening the IDE, first thing you have to do is to login into your account in the software.



# PROGRAMS

## 1. Toggle an LED

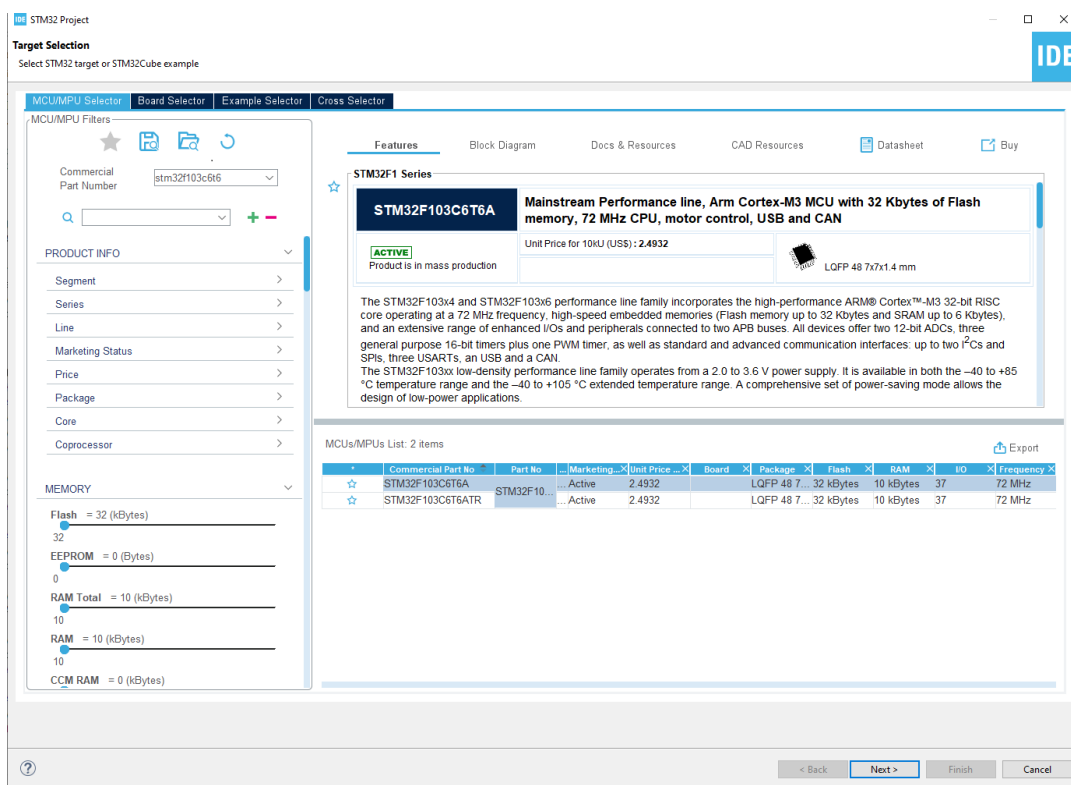
**Aim:** To toggle an LED using Ancitboard

**Software Requirements:** STM32CubeIDE

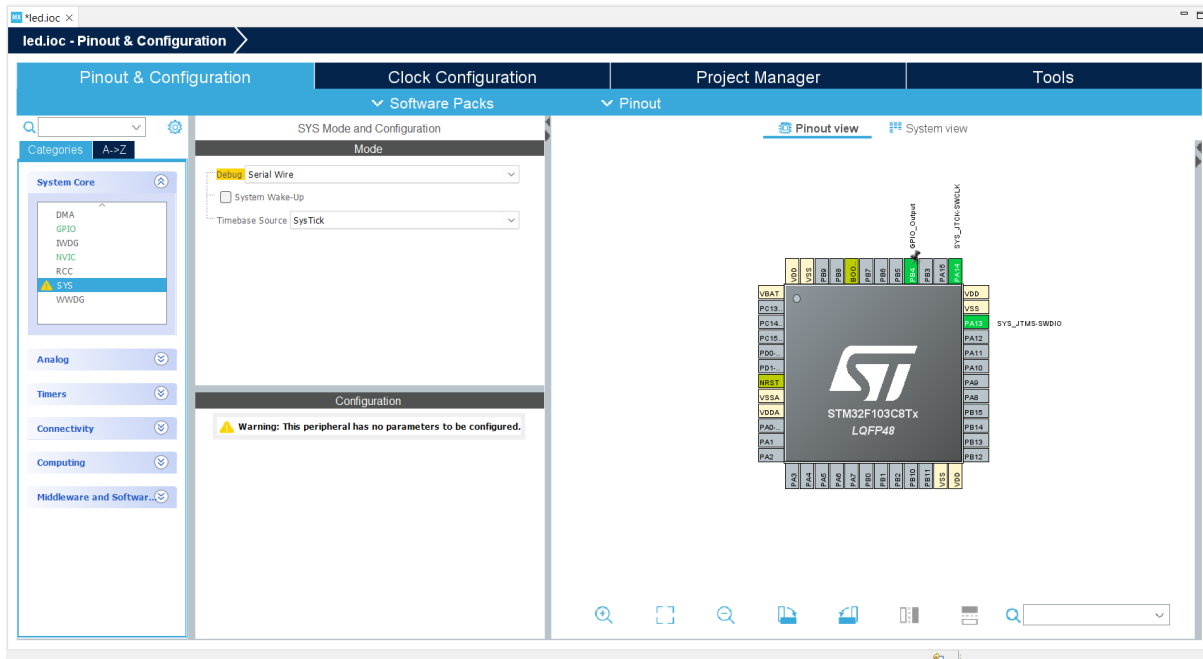
**Hardware Requirements:** Ancit STM32 board

**Procedure:**

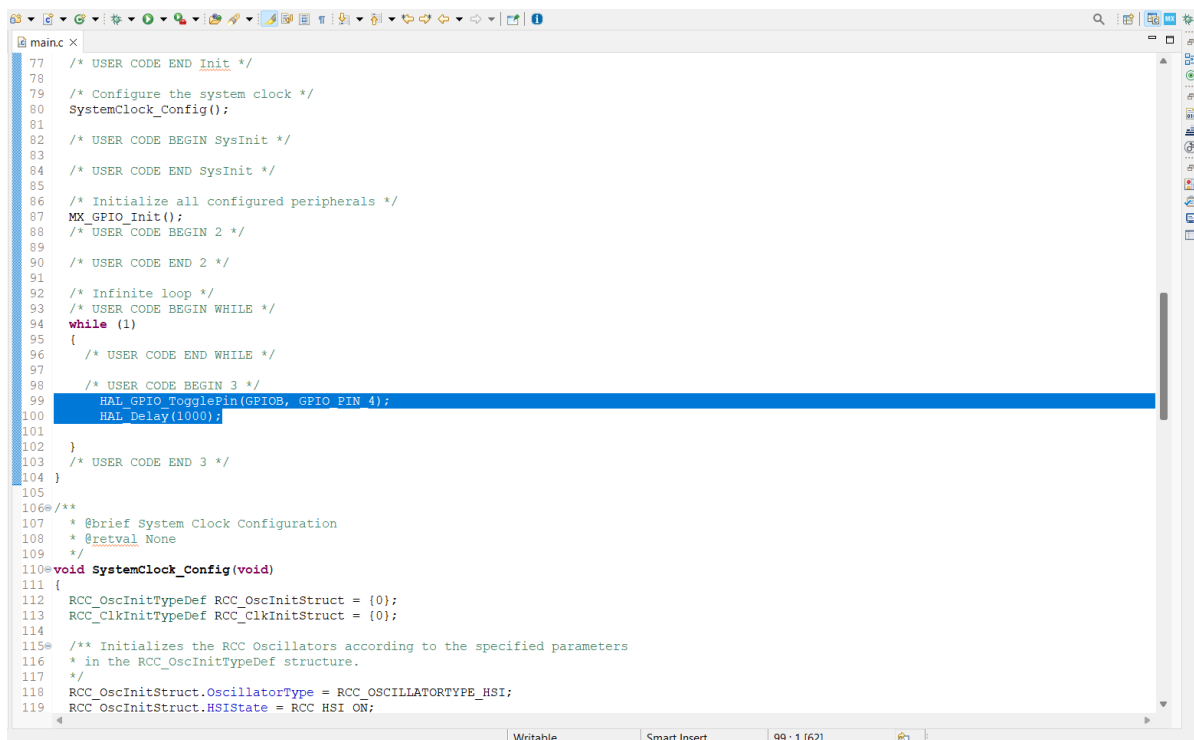
1. Open STM32CubeIDE, go to file -> new -> STM32 project.
2. Now you will get target selection window in that select part number as STM32F103C6T6A and select the MCU and click on next and then give the project name and click on finish.



3. Then you will be getting the .ioc file in which you can configure the pins of the STM32 board.
4. Now in the Pinout & Configurations, go to system core -> SYS -> select Serial wire.
5. Now in the pinout view select PB4(LED) as GPIO\_Output.
6. Then save the file and then you will get the generated main.c file.



7. Now in the main.c, complete the code as given in the image below using HAL functions.



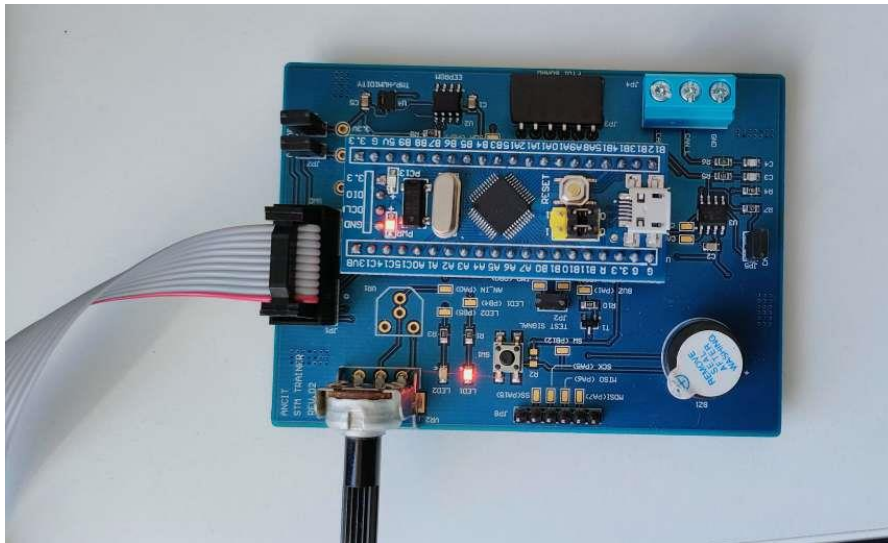
8. After writing the code click on build symbol  and verify your code.

9. Now connect the Ancit board and click on the run symbol. 



**Output:**

You will get the output as LED(PB4) will toggle for every one second.



**Note:** you can repeat the same program for another LED(PB5) and you can also try to toggle both LED's

## 2. Switch Operation

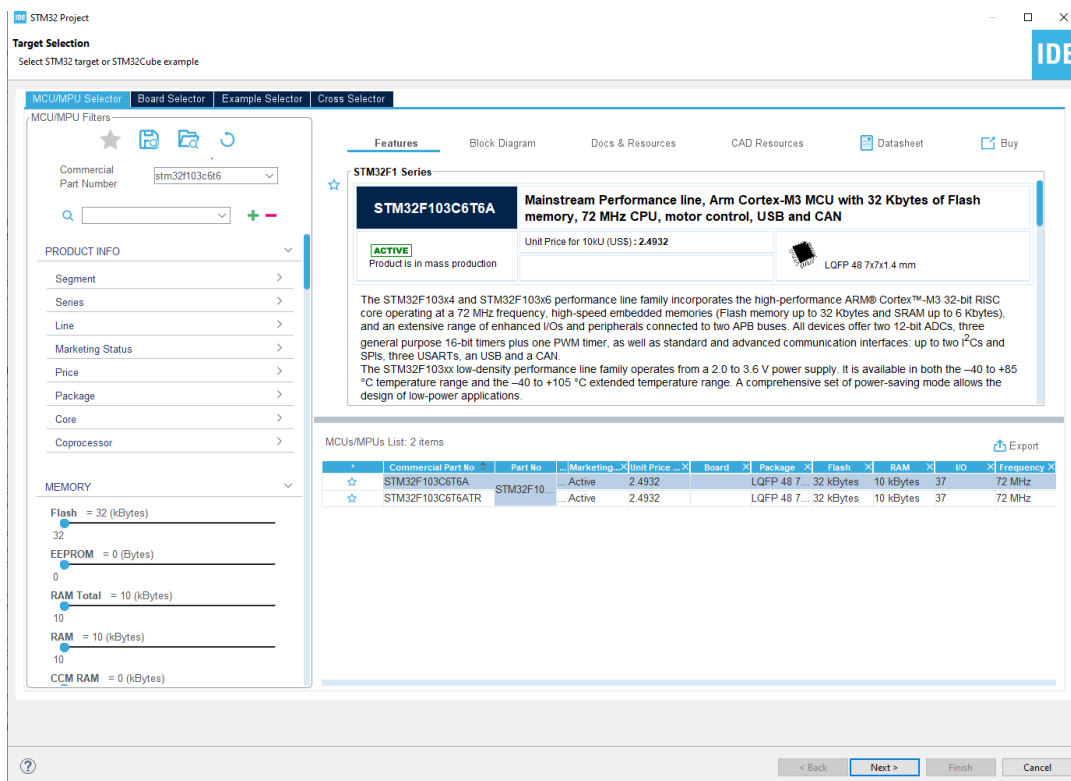
**Aim:** To turn ON or OFF an LED using Switch

**Software Requirements:** STM32CubeIDE

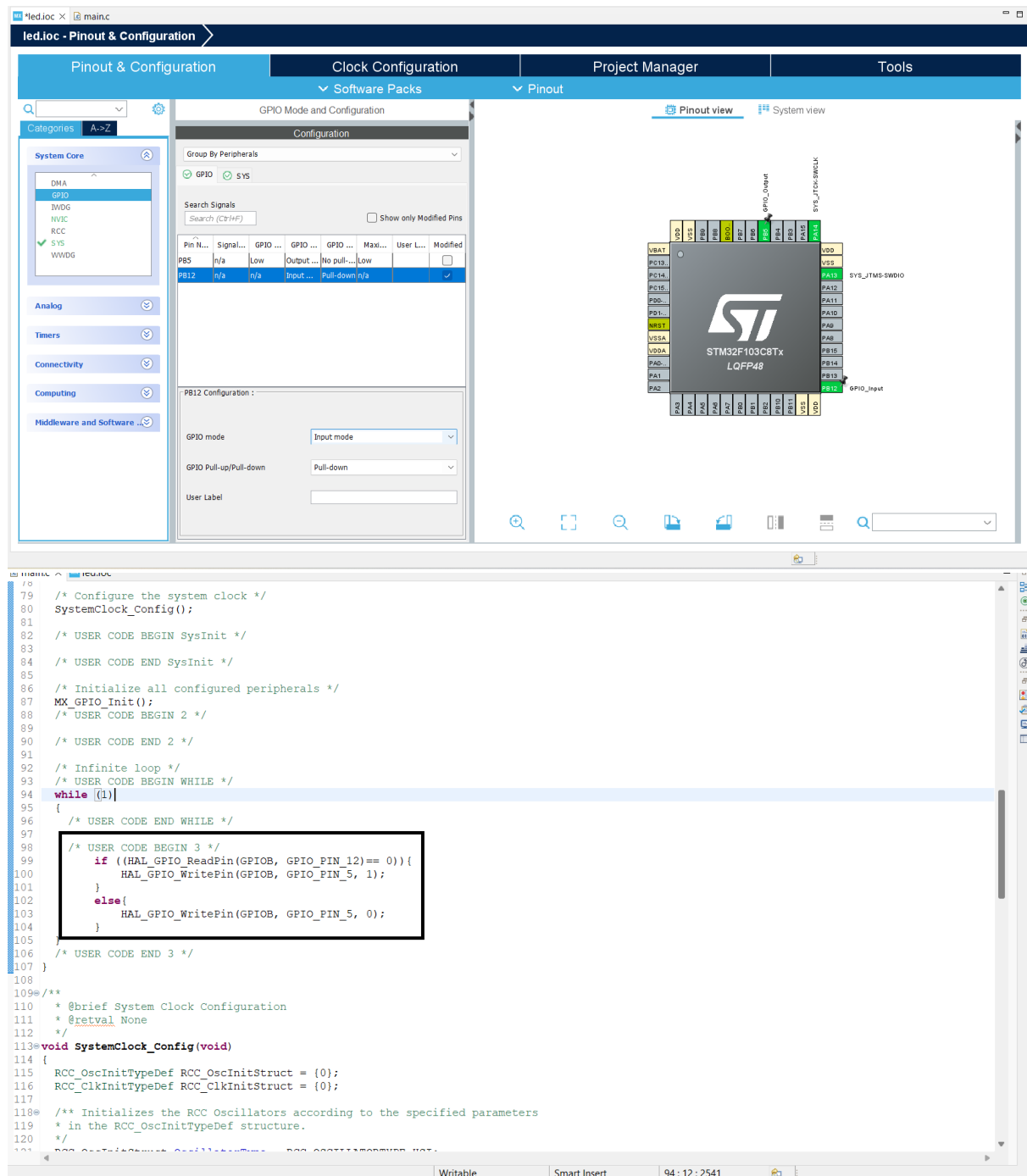
**Hardware Requirements:** Ancit STM32 board

**Procedure:**

1. Open STM32CubeIDE, go to file -> new -> STM32 project.
2. Now you will get target selection window in that select part number as STM32F103C6T6A and select the MCU and click on next and then give the project name and click on finish.



3. Then you will be getting the .ioc file in which you can configure the pins of the STM32 board.
4. Now in the Pinout & Configurations, go to system core -> SYS -> select Serial wire.
5. Now select PB12(Switch) as GPIO\_Input and PB5(LED) as GPIO\_Output in Pinout view.
6. In system core -> GPIO -> click on PB12 and in GPIO Pull-up/Pull-down select Pull-Up.  
note: By selecting Pull-down the switch initial state will be 0.
7. After giving all these configurations click on save, then main.c will be generated.
8. Now in the main.c, complete the code as given in the image using HAL functions.



9. After writing the code click on build symbol  and verify your code.

10. Now connect the Ancit board and click on the run symbol. 

### Output:

You will be getting the output as when Switch is pressed the LED will blink.

### 3. Breathing of LED's (PWM)

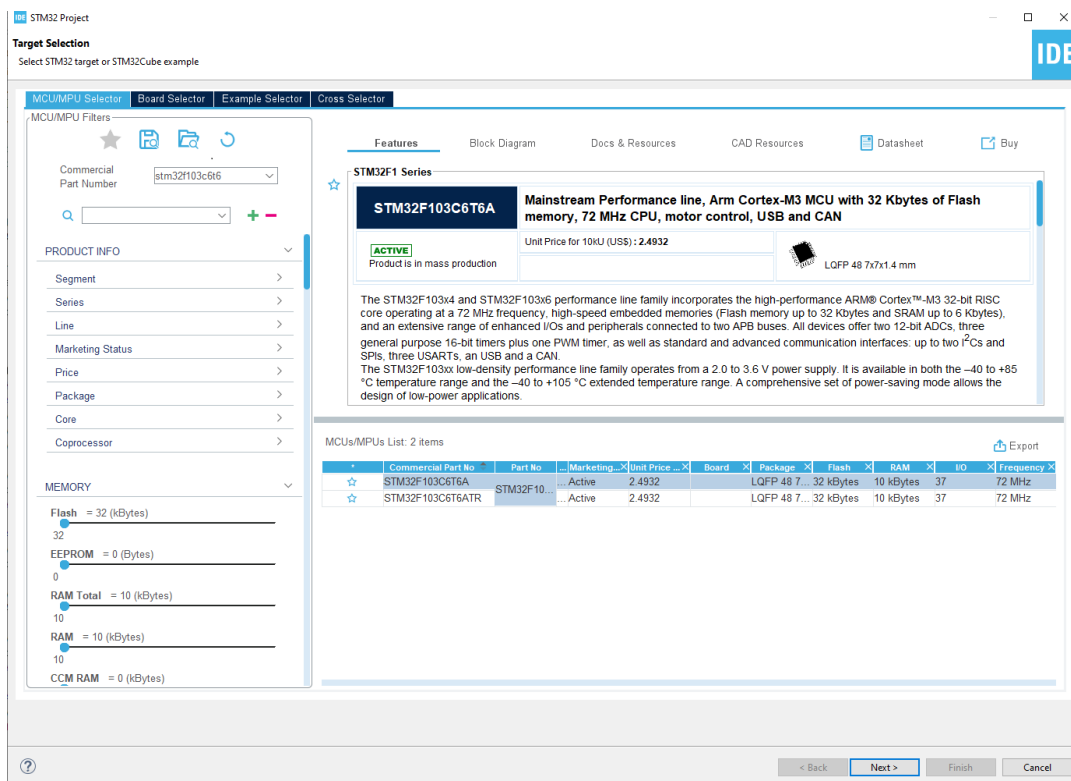
**Aim:** By using PWM increase and decrease the intensity of the LED's

**Software Requirements:** STM32CudeIDE

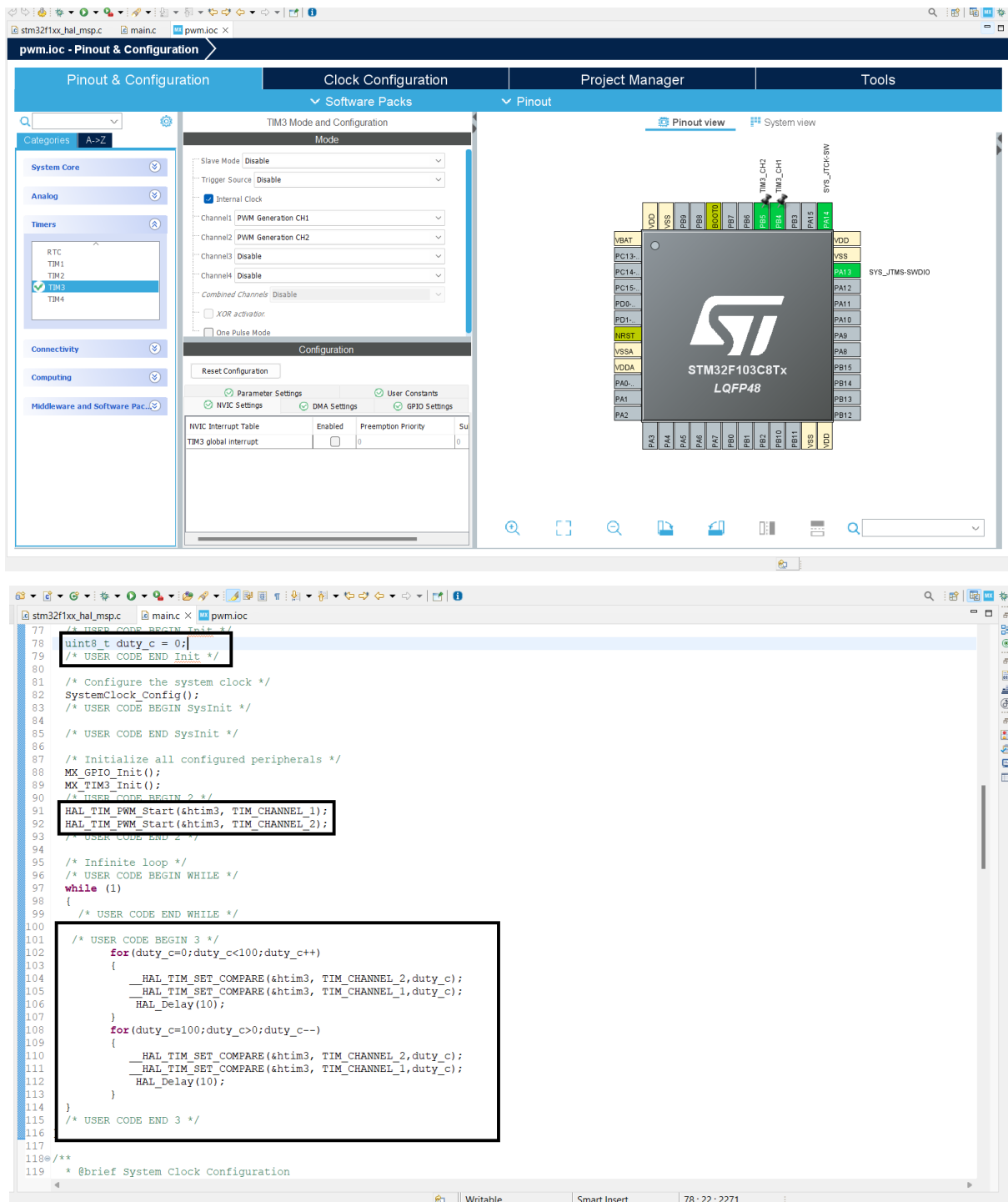
**Hardware Requirements:** Ancit STM32 board

**Procedure:**

1. Open STM32CubeIDE, go to file -> new -> STM32 project.
2. Now you will get target selection window in that select part number as STM32F103C6T6A and select the MCU and click on next and then give the project name and click on finish.



3. Then you will be getting the .ioc file in which you can configure the pins of the STM32 board.
4. Now in the Pinout & Configurations, go to system core -> SYS -> select Serial wire.
5. Select PB4 and PB5 as TIM3\_CH1 and TIM3\_CH2 in Pinout view.
6. Go to timers and select TIM 3 and in mode select Internal clock and in channel 1 select PWM Generation CH1 in the dropdown and in channel 2 select PWM Generation CH2.
7. After giving all these configurations click on save, then main.c will be generated.
8. Now in the main.c, complete the code as given in the image below using HAL functions.



9. After writing the code click on build symbol  and verify your code.

10. Now connect the Ancit board and click on the run symbol. 

### Output:

You can see the intensity of LED's goes on increasing and decreasing continuously.

## 4. ADC Operation

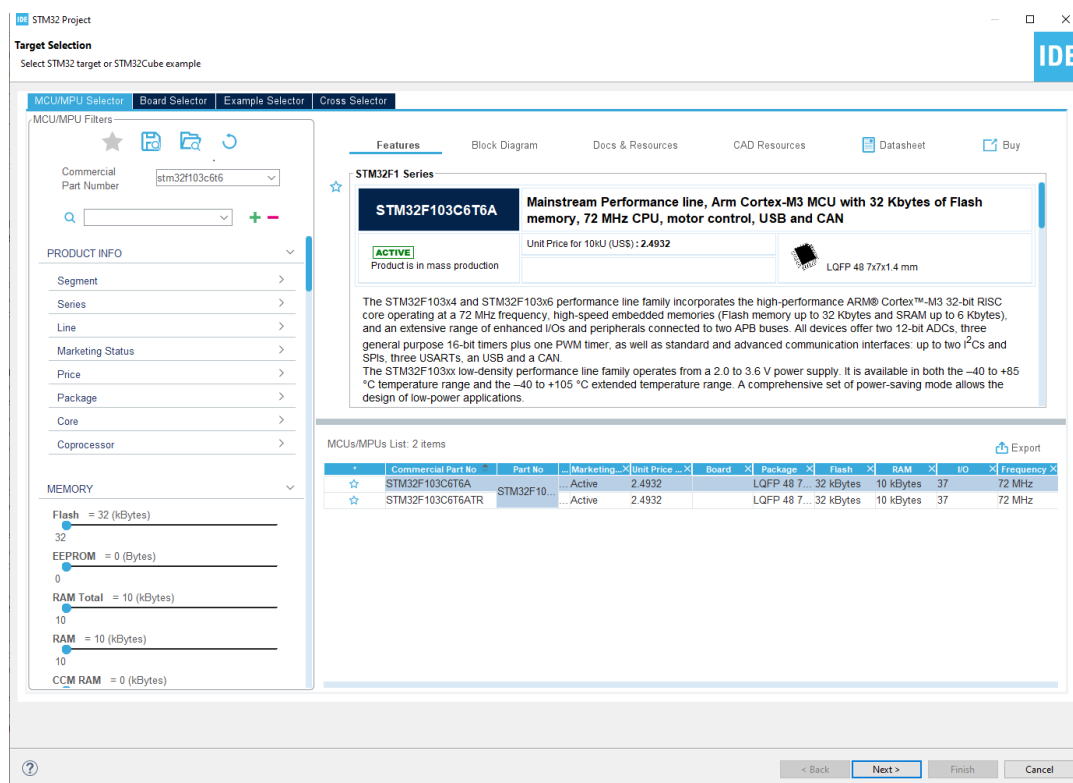
**Aim:** By using potentiometer increase and decrease the intensity of the LED's

**Software Requirements:** STM32CubeIDE

**Hardware Requirements:** Ancit STM32 board

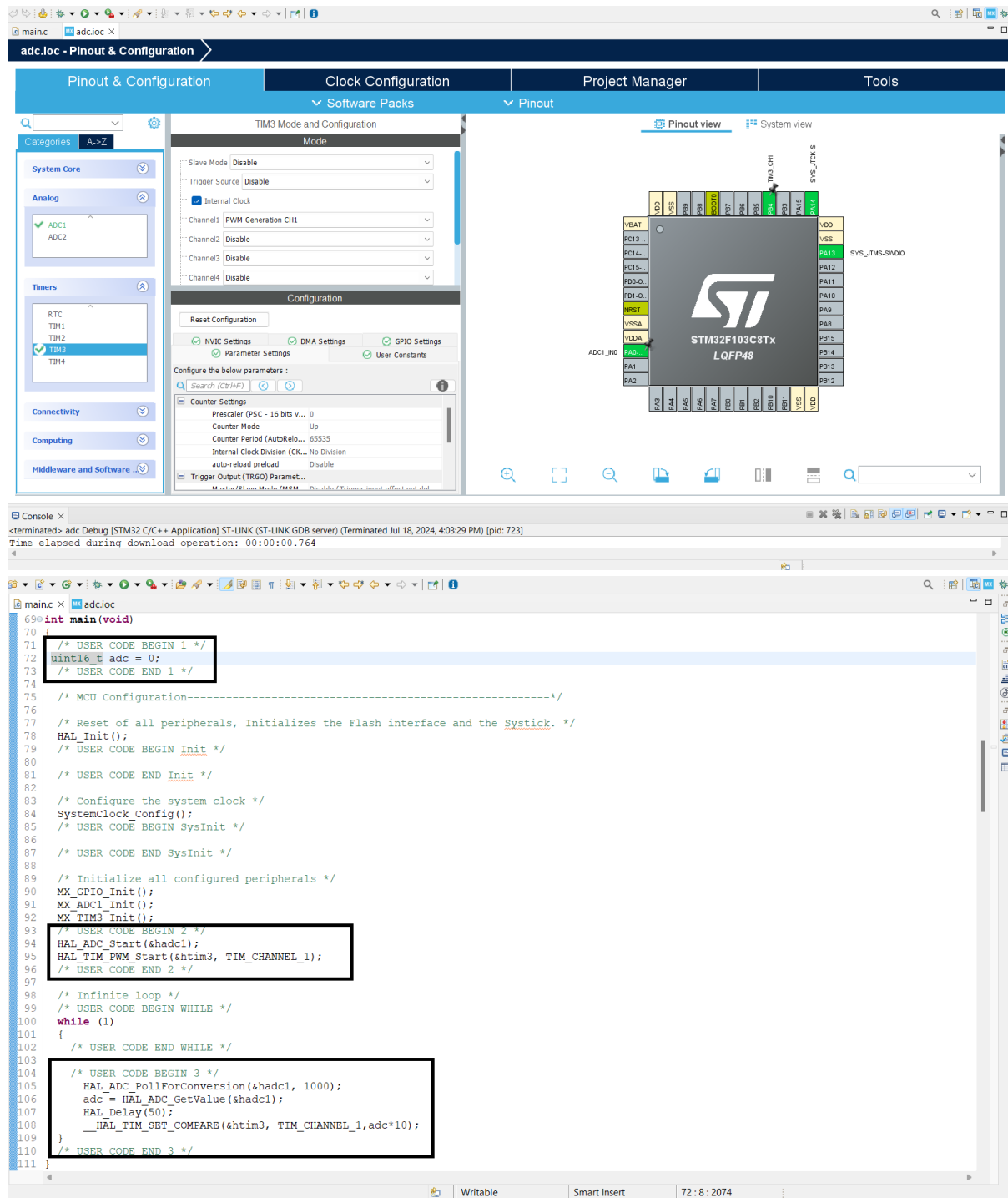
**Procedure:**

1. Open STM32CubeIDE, go to file -> new -> STM32 project.
2. Now you will get target selection window in that select part number as STM32F103C6T6A and select the MCU and click on next and then give the project name and click on finish.



3. Then you will be getting the .ioc file in which you can configure the pins of the STM32 board.
4. Now in the Pinout & Configurations, go to system core -> SYS -> select Serial wire.
5. Select PA0(Potentiometer) as ADC1\_IN0 and select PB4(LED) as TIM3\_CH1 in Pin out view.
6. Now go to timer in that go to TIM3 and click internal clock and in channel 1 dropdown select PWM Generation CH1.
7. After giving all these configurations click on save, then main.c will be generated.
8. Now in the main.c, complete the code as given in the image below using HAL functions.
9. After writing the code click on build symbol  and verify your code.





**Output:**

You will be getting the out as when you increase the potentiometer the LED intensity increases and intensity decreases when you decrease the potentiometer.

## 5. UART Working

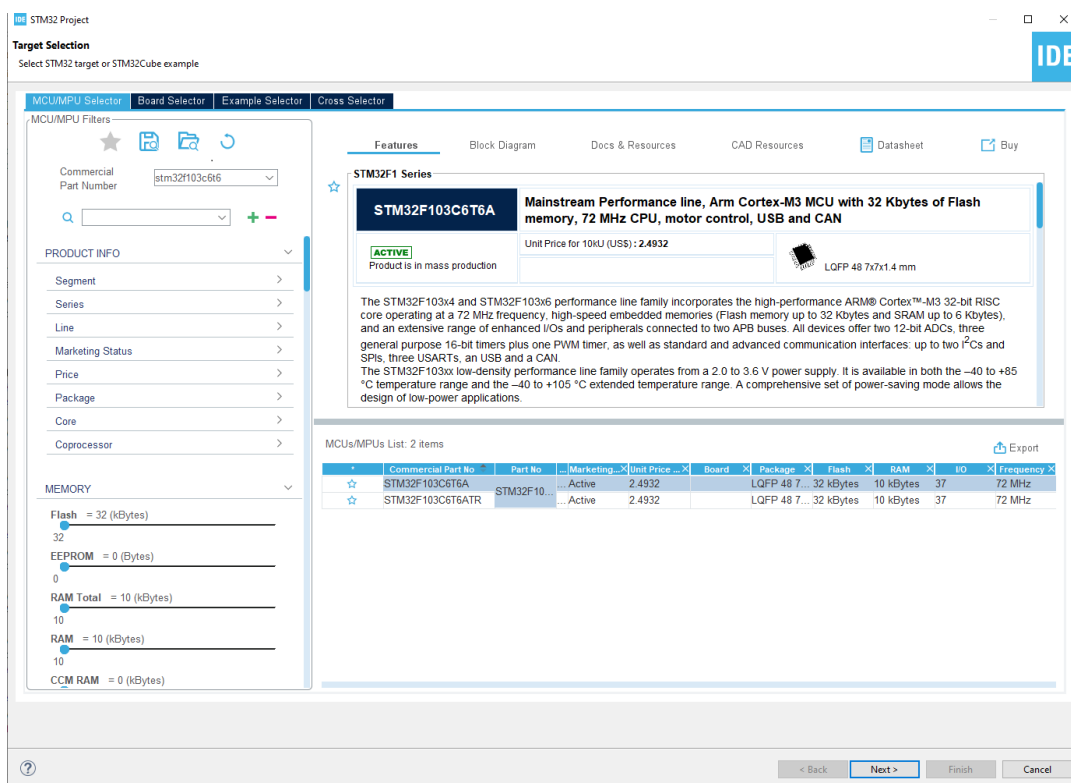
**Aim:** Use potentiometer and transmit that data through uart.

**Software Requirements:** STM32CubeIDE

**Hardware Requirements:** Ancit STM32 board

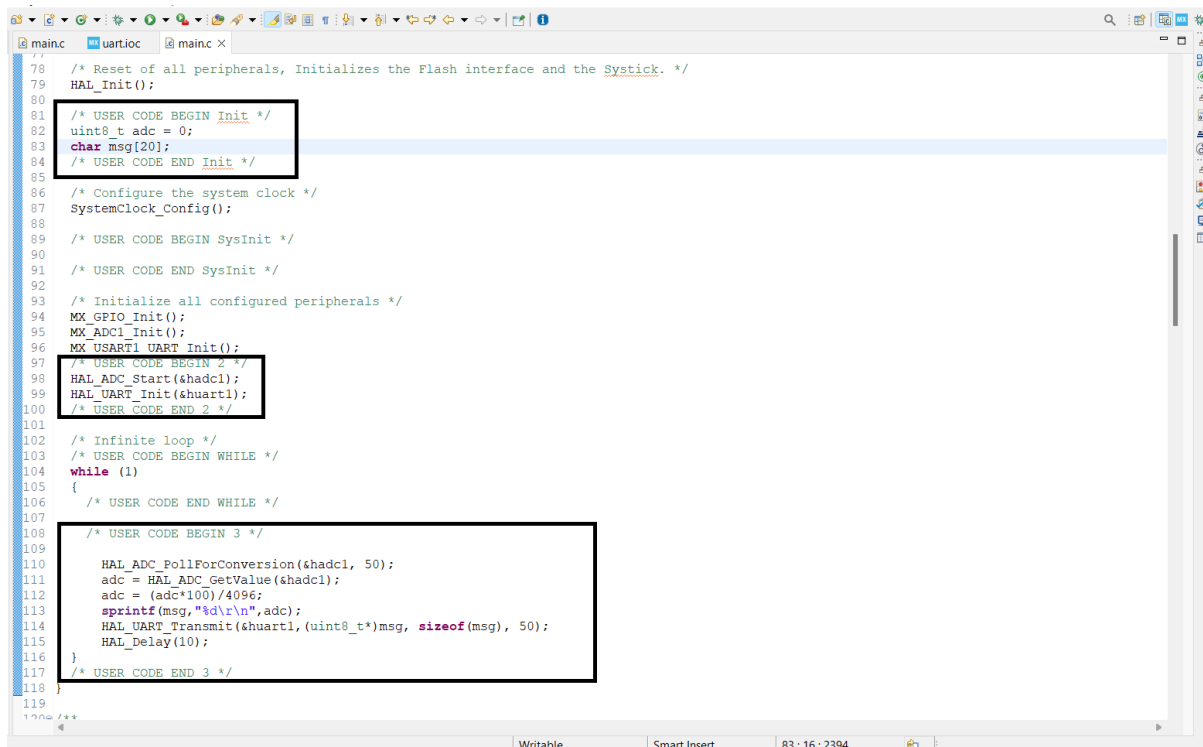
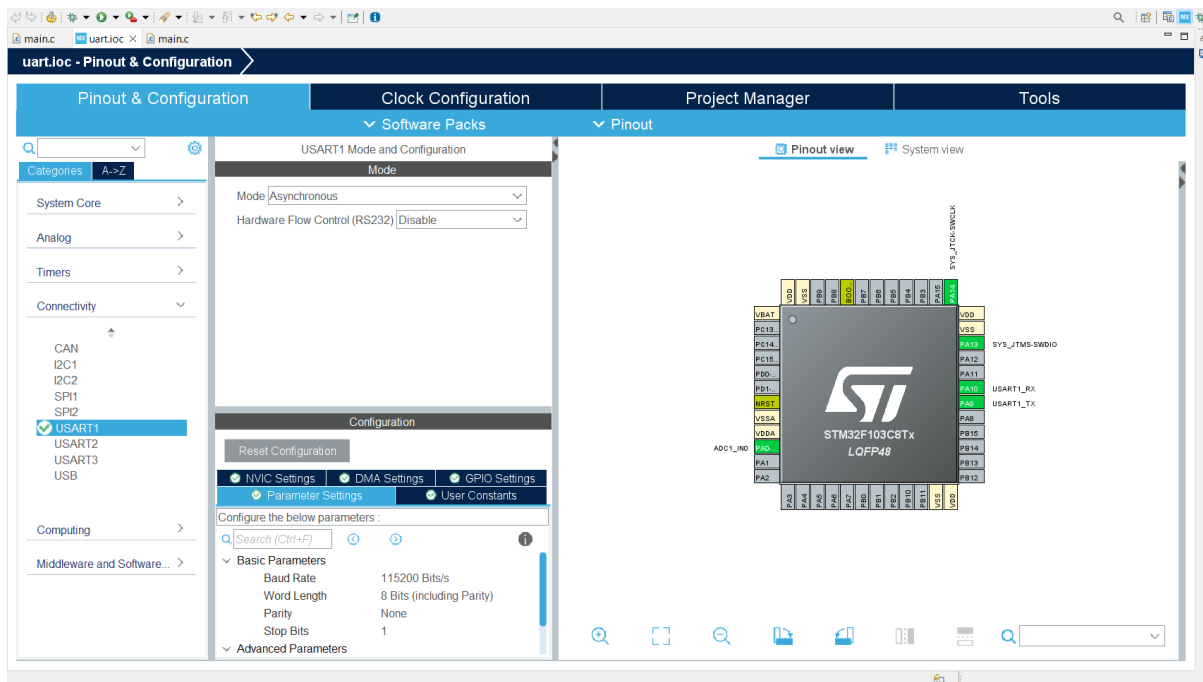
**Procedure:**

1. Open STM32CubeIDE, go to file -> new -> STM32 project.
2. Now you will get target selection window in that select part number as STM32F103C6T6A and select the MCU and click on next and then give the project name and click on finish.



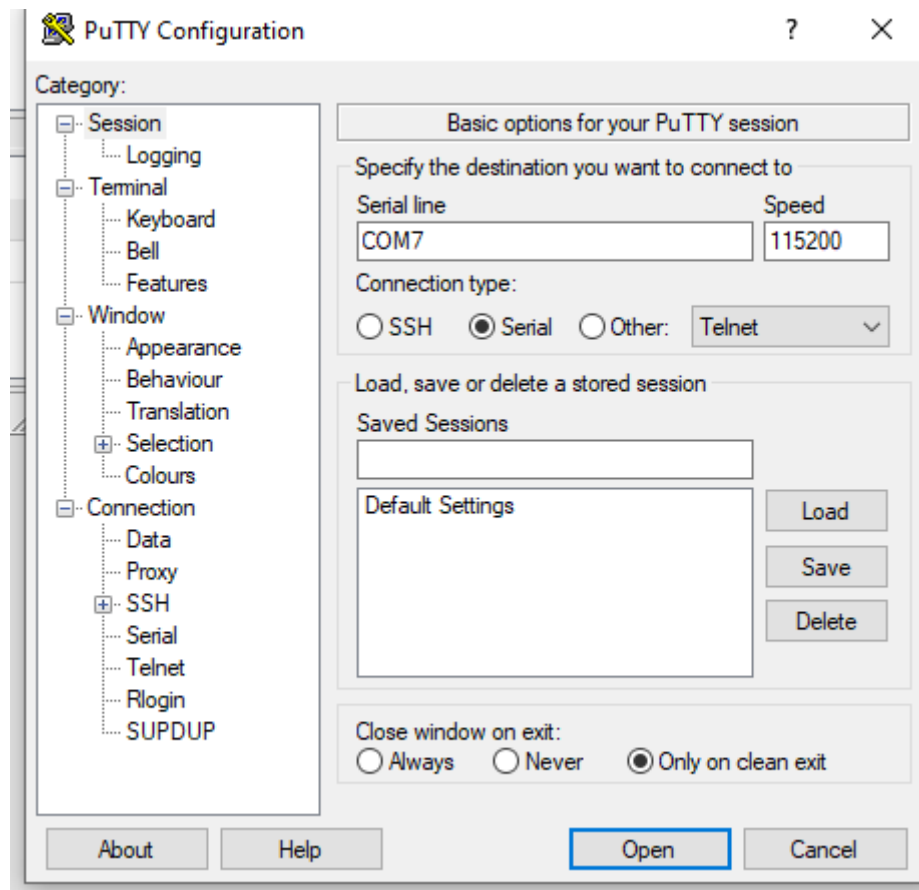
3. Then you will be getting the .ioc file in which you can configure the pins of the STM32 board.
4. Now in the Pinout & Configurations, go to system core -> SYS -> select Serial wire.
5. Select PA0 as ADC1\_IN0 in Pinout view and now in categories go to connectivity and select UART1 and mode as asynchronous and in NVIC settings enable UART1 global interrupts.
6. After giving all these configurations click on save, then main.c will be generated.
7. Now in the main.c, complete the code as given in the image below using HAL functions.
8. After writing the code click on build symbol  and verify your code.

9. Now connect the Ancit board and click on the run symbol.



10. Now to see the data connect the FTDI to the stm32 board and open PUTTY software.

11. In putty select connection type as serial and board rate as 115200, and for the serial line you can go to your device manager and you can find respective COM ports there and give that number here in the serial line.
12. Click on open and start changing the values of pot.



**Output:**

You will get the potentiometer values on the putty window.

## 6. Timer Interrupt

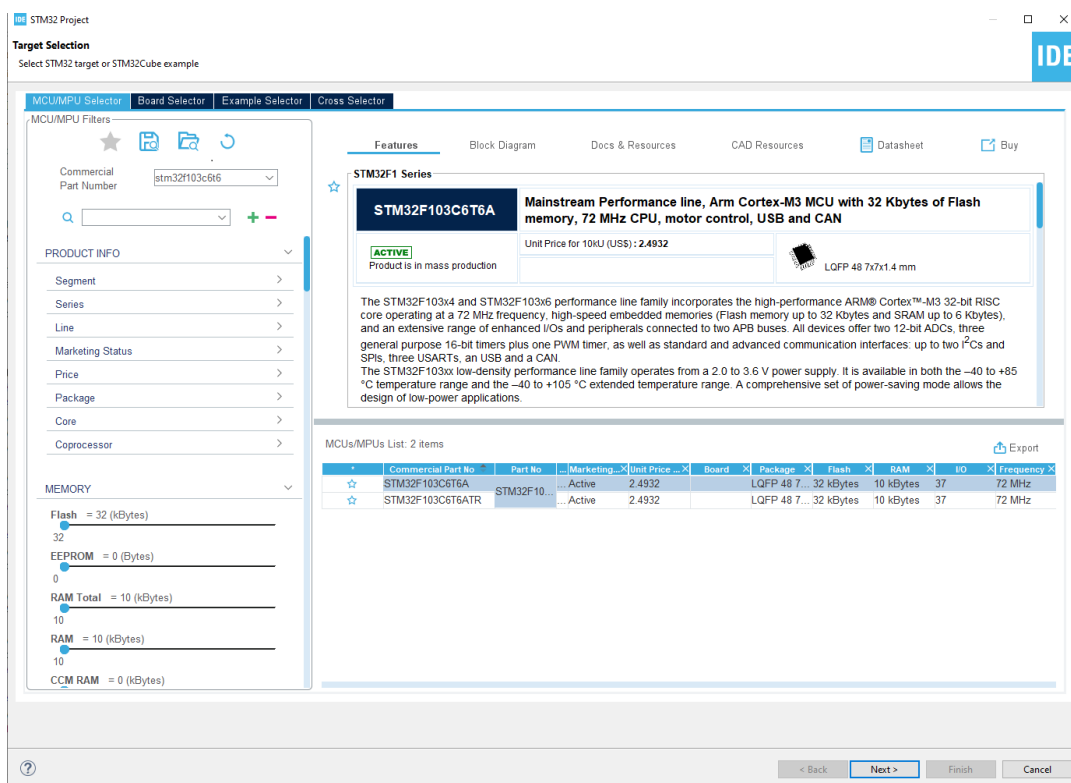
**Aim:** Blinking of Led using Timer Interrupt in the Ancit board.

**Software Requirements:** STM32CudeIDE

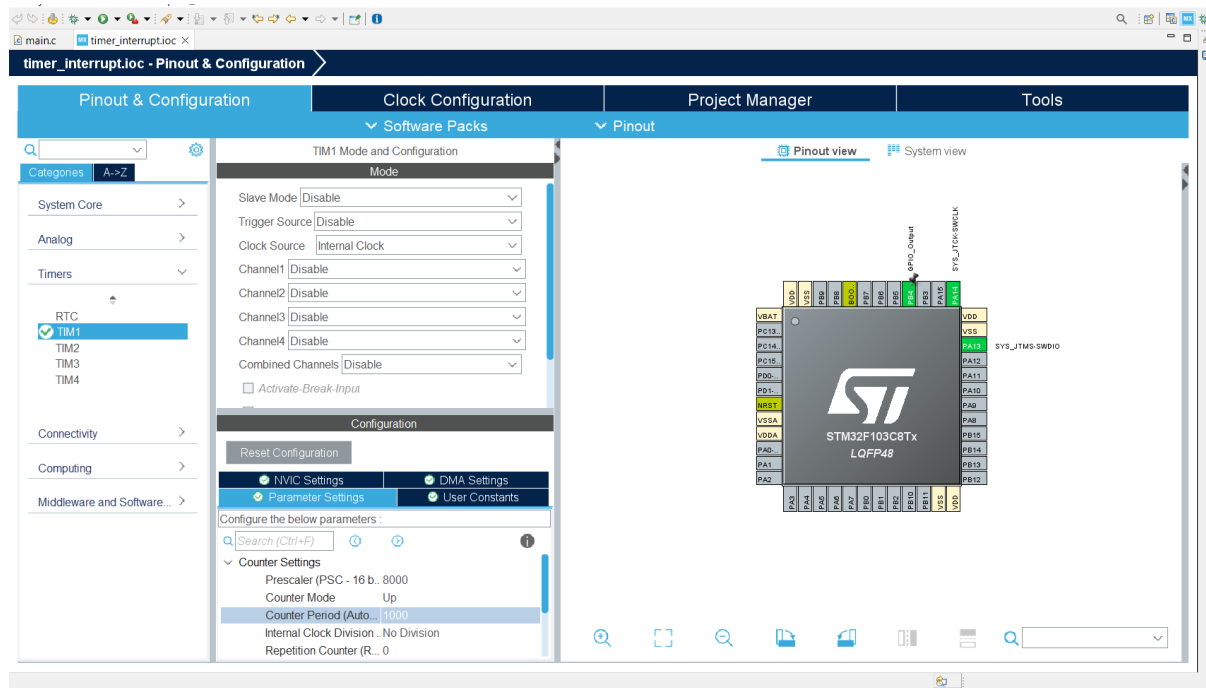
**Hardware Requirements:** Ancit STM32 board

**Procedure:**

1. Open STM32CubeIDE, go to file -> new -> STM32 project.
2. Now you will get target selection window in that select part number as STM32F103C6T6A and select the MCU and click on next and then give the project name and click on finish.



3. Then you will be getting the .ioc file in which you can configure the pins of the STM32 board.
4. Now in the Pinout & Configurations, go to system core -> SYS -> select Serial wire.
5. Click on PB4 in the Pin out view and select GPIO\_Output, and in categories go to Timers, in that click on TIM1, after that in the TIM1 configurations select internal clock.
6. Now select Prescaler as 8000 and counter period as 1000 (as it will set the timer for one second) and in NVIC settings enable TIM1 break interrupt and update interrupt.
7. After giving all these configurations click on save, then main.c will be generated.



### Need 1s.

System clock = 8MHz (you can find in clock configurations)

Choose **prescaler 8000**

So,  $\text{SYSCLK}/\text{prescaler} = 8\text{MHz}/8000 = 1000$  --- frequency

Convert into time ==  $1/\text{freq} == 1/1000 = 1\text{ms}$

Choose **counter period == 1000.**

So, we get =  $1\text{ms} \times 1000 = 1\text{sec}$

8. Now in the main.c, complete the code as given in the image below using HAL functions.

9. After writing the code click on build symbol  and verify your code.

10. Now connect the Ancit board and click on the run symbol. 

```

mainc X timer_interruptioc
170 /* USER CODE BEGIN 1 */
171
172 /* MCU Configuration-----*/
173
174 /* Reset of all peripherals, Initializes the Flash interface and the SysTick. */
175 HAL_Init();
176
177 /* USER CODE BEGIN Init */
178
179 /* USER CODE END Init */
180
181 /* Configure the system clock */
182 SystemClock_Config();
183
184 /* USER CODE BEGIN SysInit */
185
186 /* USER CODE END SysInit */
187
188 /* Initialize all configured peripherals */
189 MX_GPIO_Init();
190 MX_TIM1_Init();
191
192 /* USER CODE BEGIN 2 */
193 HAL_TIM_Base_Start_IT(&htim1);
194 /* USER CODE END 2 */
195
196 /* Infinite loop */
197 /* USER CODE BEGIN WHILE */
198 while (1)
199 {
200     /* USER CODE END WHILE */
201
202     /* USER CODE BEGIN 3 */
203 }
204 /* USER CODE END 3 */
205
206 /**
207  * @brief System Clock Configuration
208  *
209  */

```

```

mainc X timer_interruptioc
196 /* USER CODE BEGIN MX_GPIO_Init_1 */
197 /* USER CODE END MX_GPIO_Init_1 */
198
199 /* GPIO Ports Clock Enable */
200 __HAL_RCC_GPIOA_CLK_ENABLE();
201 __HAL_RCC_GPIOB_CLK_ENABLE();
202
203 /*Configure GPIO pin Output Level */
204 HAL_GPIO_WritePin(GPIOB, GPIO_PIN_4, GPIO_PIN_RESET);
205
206 /*Configure GPIO pin : PB4 */
207 GPIO_InitStruct.Pin = GPIO_PIN_4;
208 GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
209 GPIO_InitStruct.Pull = GPIO_NOPULL;
210 GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
211 HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);
212
213 /* USER CODE BEGIN MX_GPIO_Init_2 */
214 /* USER CODE END MX_GPIO_Init_2 */
215
216
217 /* USER CODE BEGIN 4 */
218 weak void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
219 /*this the called function for the timer you can copy this method from
220 Drivers -> SYM32F1xx_HAL_Driver -> src -> stm32f1xx_hal_tim.c */
221 {
222     HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_4);
223 }
224
225 /* USER CODE END 4 */
226
227
228 /**
229  * @brief This function is executed in case of error occurrence.
230  * @retval None
231  */
232 void Error_Handler(void)
233 {
234     /* USER CODE BEGIN Error_Handler_Debug */
235     /* User can add his own implementation to report the HAL error return state */
236     __disable_irq();
237     while (1)
238     {

```

## Output:

you should get the led 1(PB4) blinking for every one second.



## 7. CAN TX and RX

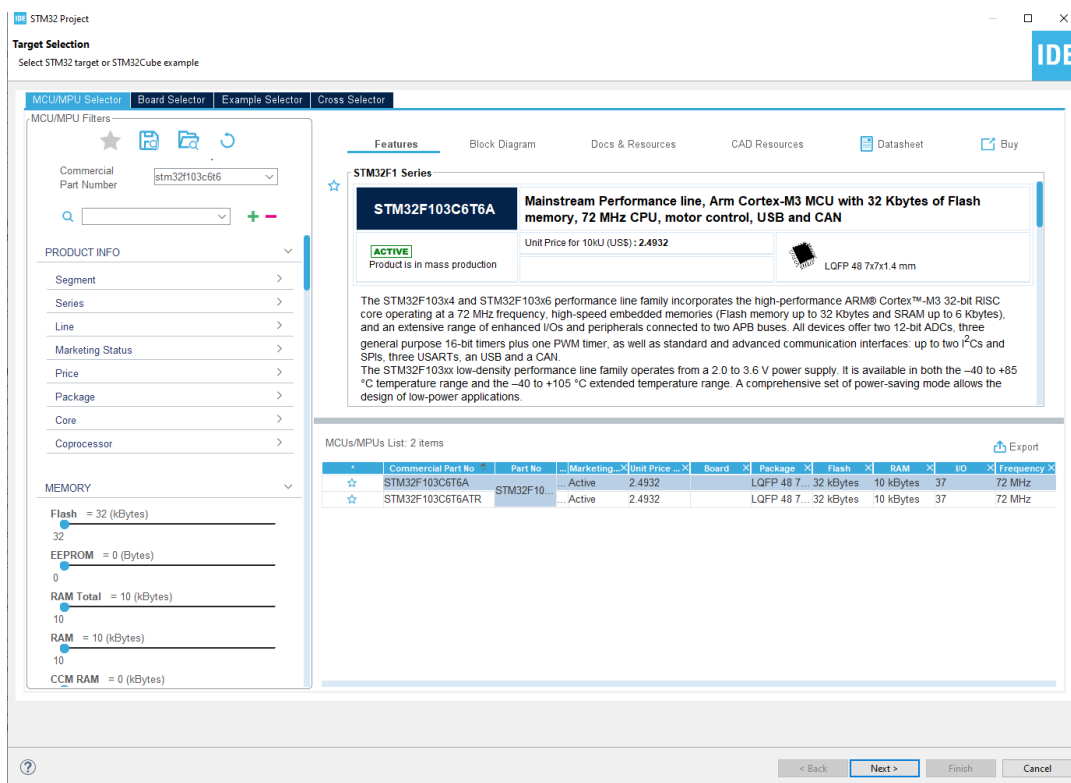
**Aim:** Check the CAN transmit and Receive message in the Ancit board.

**Software Requirements:** STM32CudeIDE

**Hardware Requirements:** Ancit STM32 board

**Procedure:**

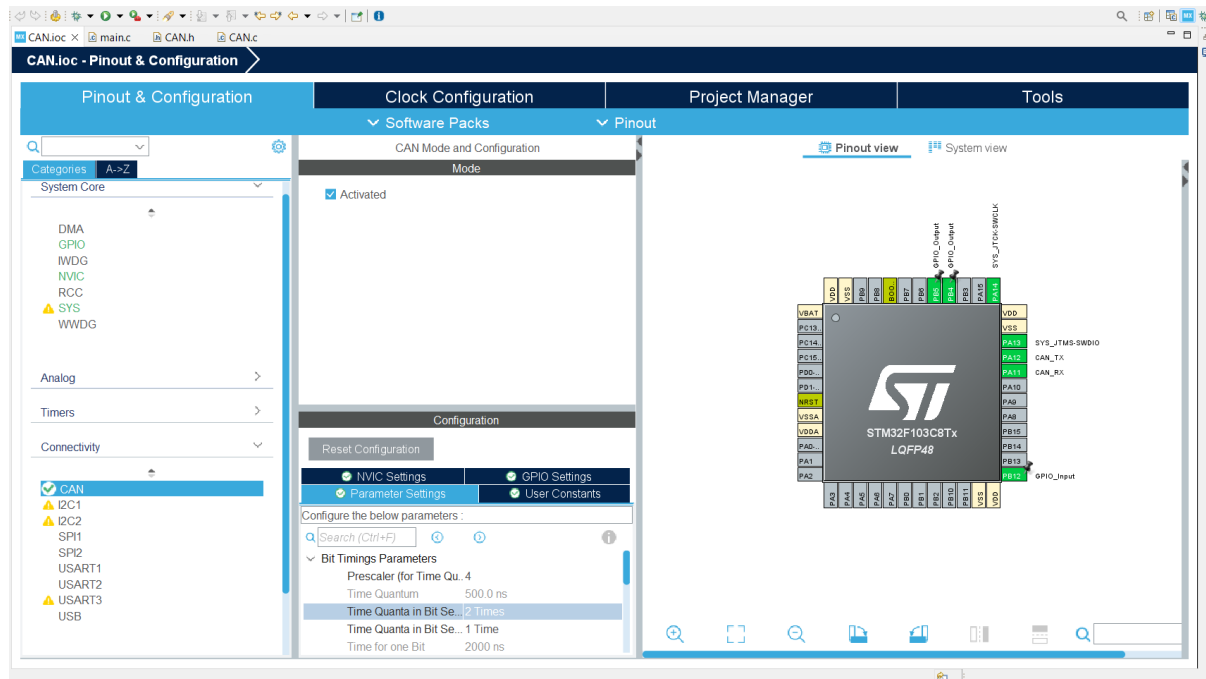
1. Open STM32CubeIDE, go to file -> new ->STM32 project.
2. Now you will get target selection window in that select part number as STM32F103C6T6A and select the MCU and click on next and then give the project name and click on finish.



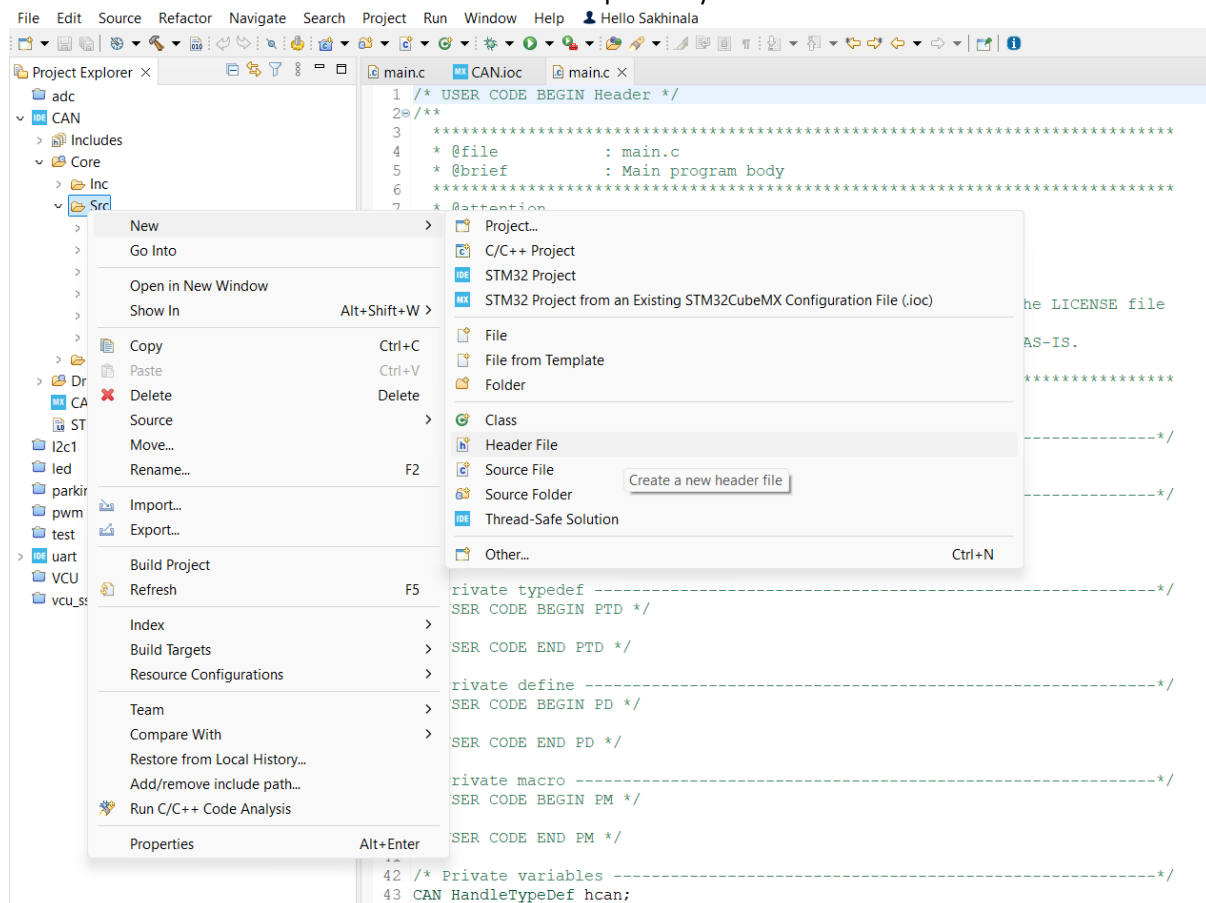
3. Then you will be getting the .ioc file in which you can configure the pins of the STM32 board.
4. Now in the Pinout & Configurations, go to system core -> SYS -> select Serial wire.
5. Select PB5 and PB4 as the GPIO\_Output and PB12 as the GPIO\_Input in Pinout view, now go to categories and in connectivity select CAN and click activate and in the NVIC settings enable TX interrupt and RX0 interrupt and in parameters give prescaler as 16 and in Time Quanta Segment 1 as 2 times.
6. In GPIO categories select PB12 as pull-up.

7. After giving all these configurations click on save, then main.c will be generated.

8. Now in the main.c, complete the code as given in the image below using HAL functions.



9. Once the main.c is generated go to core -> src right click on it go to new and create the new header file and .c file with names CAN.h and CAN.c respectively.



10. After creating that copy can.c and can.h codes into their respective files.

CAN.c

```
#include "CAN.h"
#include "main.h"

extern CAN_HandleTypeDef hcan;
can_struct_t myCan;

// Starts all the modules, Both Tx and Rx part
void ancit_can_start(void)
{
    /* IMPORTANT NOTE
       The following line has to be copied into stm32f1xx_hal_can.c file
       before the comment // Wait initialisation acknowledge
       else, HAL_CAN_Init will hang...

       CLEAR_BIT(hcan->Instance->MCR, CAN_MCR_SLEEP);
    */

    ancit_can_init_rx_filter();
    HAL_CAN_Start(&hcan);

    // Enable Rx interrupts.
    if (HAL_CAN_ActivateNotification(&hcan, CAN_IT_RX_FIFO0_MSG_PENDING) != HAL_OK)
    {
        Error_Handler();
    }
}

// Sets up the rx_filter
void ancit_can_init_rx_filter(void)
{
    CAN_FilterTypeDef sFilterConfig;

    sFilterConfig.FilterBank = 0;
    sFilterConfig.FilterMode = CAN_FILTERMODE_IDMASK;
    sFilterConfig.FilterScale = CAN_FILTERSCALE_32BIT;
    sFilterConfig.FilterIdHigh = 0x0000;
    sFilterConfig.FilterIdLow = 0x0000;
    sFilterConfig.FilterMaskIdHigh = 0x0000;
    sFilterConfig.FilterMaskIdLow = 0x0000;
    sFilterConfig.FilterFIFOAssignment = CAN_RX_FIFO0;
    sFilterConfig.FilterActivation = ENABLE;
    sFilterConfig.SlaveStartFilterBank = 14;

    if (HAL_CAN_ConfigFilter(&hcan, &sFilterConfig) != HAL_OK)
    {
        /* Filter configuration Error */
        Error_Handler();
    }

    // init Rx flag to 0;
    myCan.rx_message_flag = 0;
}
```

```

}
// Rx Message Interrupt handler
void HAL_CAN_RxFifo0MsgPendingCallback(CAN_HandleTypeDef *hcan)
{
    //HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_5);
    if (HAL_CAN_GetRxMessage(hcan, CAN_RX_FIFO0, &myCan.RxHeader, myCan.RxData) != HAL_OK)
    {
        Error_Handler();
    }
    else
    {
        myCan.rx_message_flag = 1;
    }
}

void ancit_can_check_for_rx(void)
{
    if (myCan.rx_message_flag == 1)
    {
        myCan.rx_message_flag = 0;
        // message received, take action here...
        if (myCan.RxHeader.StdId == 0x07)
        {
            HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_4);
            HAL_Delay(100);
        }
    }
}

void ancit_can_tx_message(void)
{
    myCan.TxHeader.IDE = CAN_ID_STD;
    myCan.TxHeader.StdId = 0x08;
    myCan.TxHeader.RTR = CAN_RTR_DATA;
    myCan.TxHeader.DLC = 1;
    myCan.TxData[0]=1;

    // Inserting call functions from DBC generated file.
    if (HAL_CAN_AddTxMessage(&hcan, &myCan.TxHeader, myCan.TxData, &myCan.TxMailbox) !=
    HAL_OK)
    {
        //HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_5);
        Error_Handler();
    }
}

void ancit_can_wait_for_switch_press(void)
{
    uint8_t switch_status = 0;
    static uint8_t wait_for_falling_edge = 1;
    // switch_status = HAL_GPIO_ReadPin(SW1_GPIO_Port, SW1_Pin);
    switch_status = HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_12);
    if (switch_status == 0)
    {

```

```

// switch debounce delay
    HAL_Delay(25);
//switch_status = HAL_GPIO_ReadPin(SW1_GPIO_Port, SW1_Pin);
    switch_status = HAL_GPIO_ReadPin(GPIOB,GPIO_PIN_12);
    if (switch_status == 0)
    {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_5,!switch_status );
        if (wait_for_falling_edge == 1)
        {
            wait_for_falling_edge = 0;
            ancit_can_tx_message();
        }
    }
    else
    {
        wait_for_falling_edge = 1;
    }
}

```

## CAN.h

```

#ifndef SRC_CAN_H_
#define SRC_CAN_H_

#include "main.h"
typedef struct
{
    CAN_TxHeaderTypeDef TxHeader;
    uint8_t TxData[8];
    uint32_t TxMailbox;

    CAN_RxHeaderTypeDef RxHeader;
    uint8_t RxData[8];

    uint8_t rx_message_flag;
}can_struct_t;

extern can_struct_t myCan;

void ancit_can_start(void);
void ancit_can_init_rx_filter(void);
void ancit_can_check_for_rx(void);
void ancit_can_tx_message(void);
void ancit_can_wait_for_switch_press(void);

#endif /* SRC_CAN_H_ */

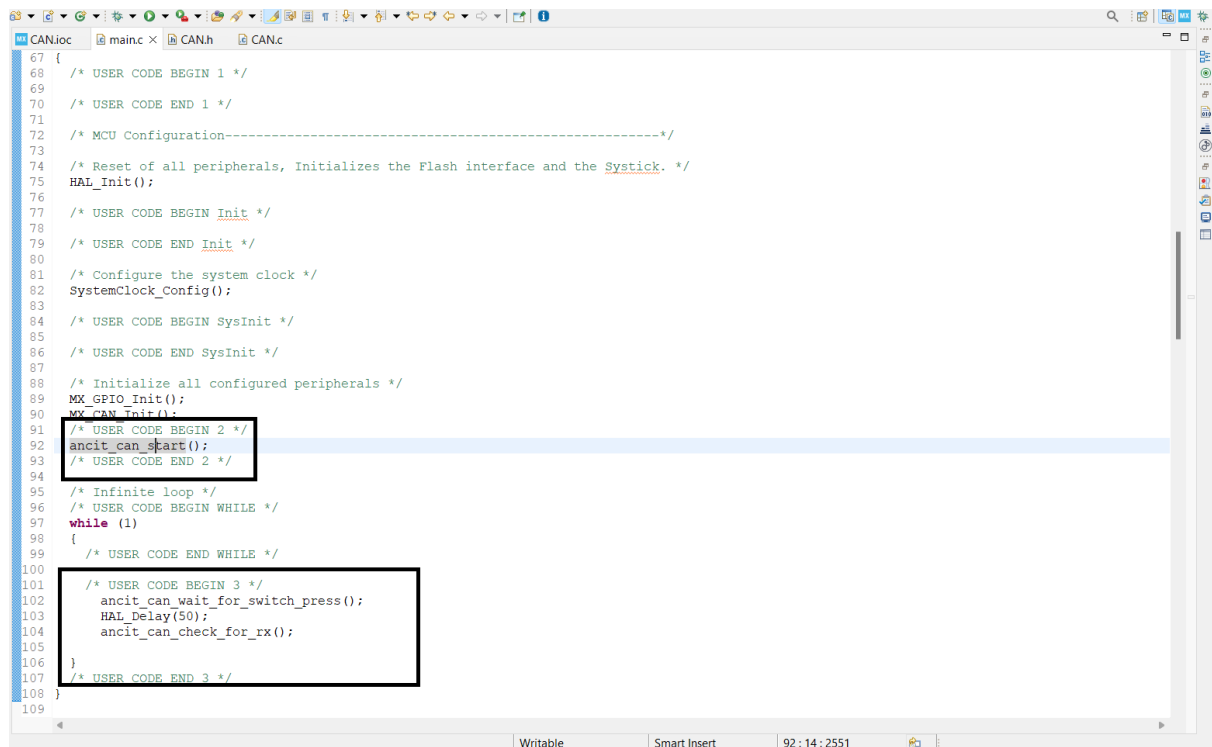
```



```

7  * @attention
8  *
9  * Copyright (c) 2024 STMicroelectronics.
10 * All rights reserved.
11 *
12 * This software is licensed under terms that can be found in the LICENSE file
13 * in the root directory of this software component.
14 * If no LICENSE file comes with this software, it is provided AS-IS.
15 *
16 *****
17 */
18 /* USER CODE END Header */
19 /* Includes -----*/
20 #include "main.h"
21
22 /* Private includes -----*/
23 /* USER CODE BEGIN Includes */
24 #include "CAN.h"
25 /* USER CODE END Includes */
26
27 /* Private typedef -----*/
28 /* USER CODE BEGIN PTD */
29
30 /* USER CODE END PTD */
31
32 /* Private define -----*/
33 /* USER CODE BEGIN PD */
34
35 /* USER CODE END PD */

```



```

67 {
68 /* USER CODE BEGIN 1 */
69
70 /* USER CODE END 1 */
71
72 /* MCU Configuration-----*/
73
74 /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
75 HAL_Init();
76
77 /* USER CODE BEGIN Init */
78
79 /* USER CODE END Init */
80
81 /* Configure the system clock */
82 SystemClock_Config();
83
84 /* USER CODE BEGIN SysInit */
85
86 /* USER CODE END SysInit */
87
88 /* Initialize all configured peripherals */
89 MX_GPIO_Init();
90 MX_CAN_Init();
91 /* USER CODE BEGIN 2 */
92 ancit_can_start();
93 /* USER CODE END 2 */
94
95 /* Infinite loop */
96 /* USER CODE BEGIN WHILE */
97 while (1)
98 {
99 /* USER CODE END WHILE */
100
101 /* USER CODE BEGIN 3 */
102 ancit_can_wait_for_switch_press();
103 HAL_Delay(50);
104 ancit_can_check_for_rx();
105 }
106 /* USER CODE END 3 */
107
108 }
109

```

11. After giving all these configurations click on save, then main.c will be generated.
12. After writing the code click on build symbol  and verify your code.
13. Now connect the Ancit board and click on the run  symbol.
14. Now connect the can high, can low, ground to the respected can connectors and verify the outputs.

### Output:

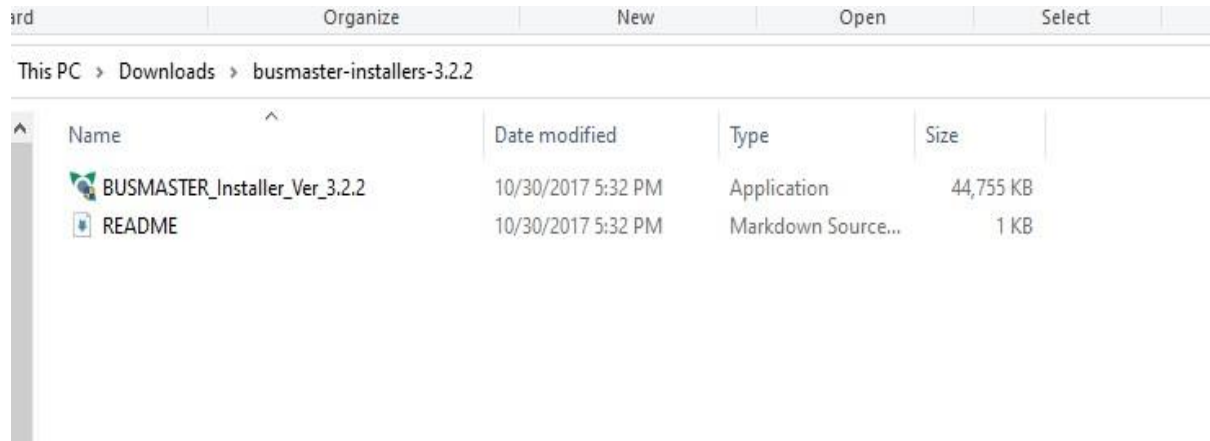
You have to check that when 0x07 id is sent the PB4 should toggle and when is press the switch can signal should transmit and the PB5 should blink.

# BUSMASTER INSTALLATION

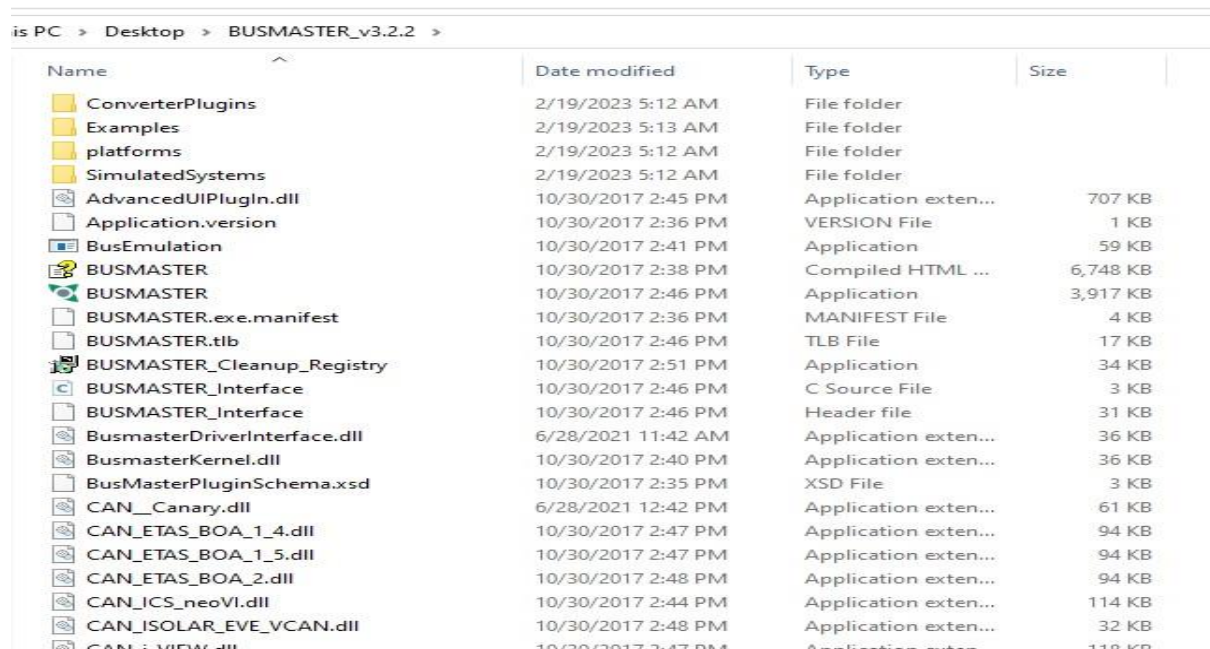
1. Open the below link:

<https://rbei-etas.github.io/busmaster/download.html>

2. Extract and double click to open (on extraction of zip file)

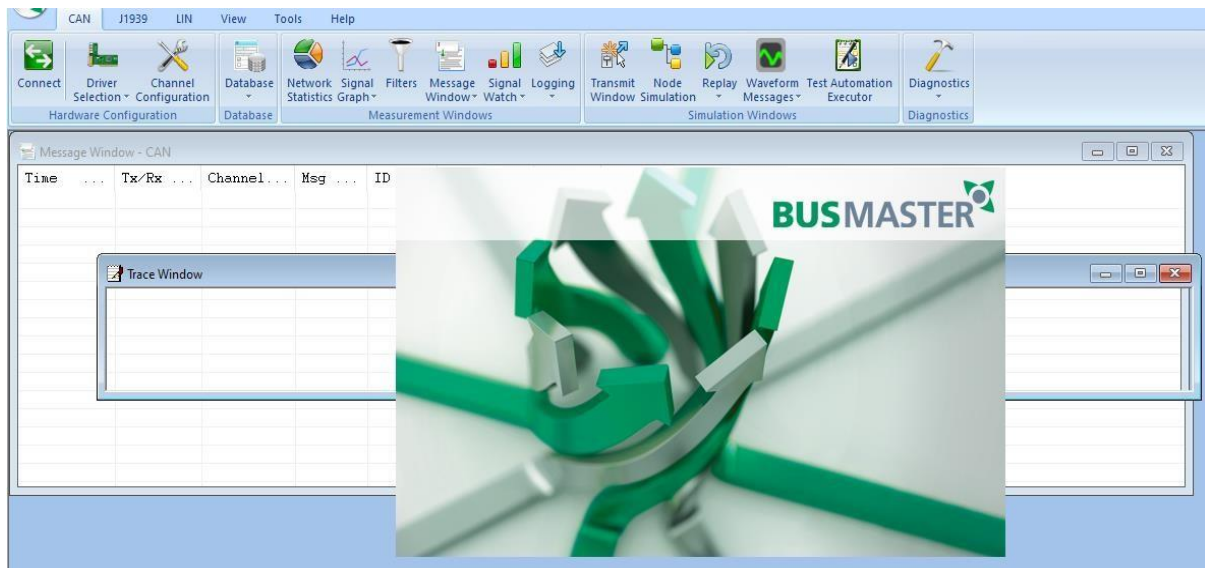


3. Check the c disk program files(x86) location OR your files saving locations find the below folder is present. (as per below image)

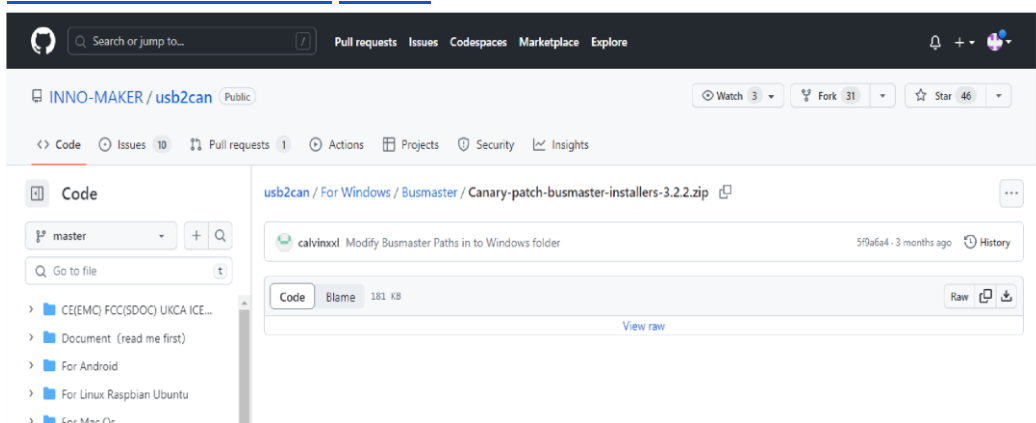


4. Search the BUSMASTER v3.2.2 & related application open. (below image)

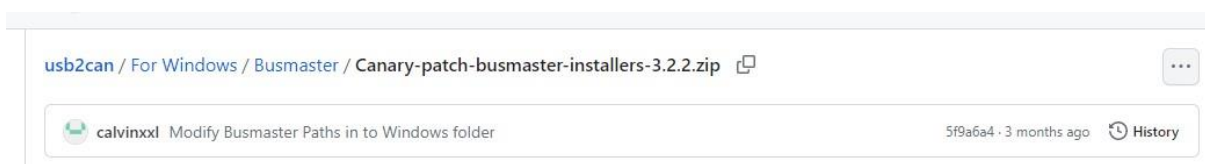




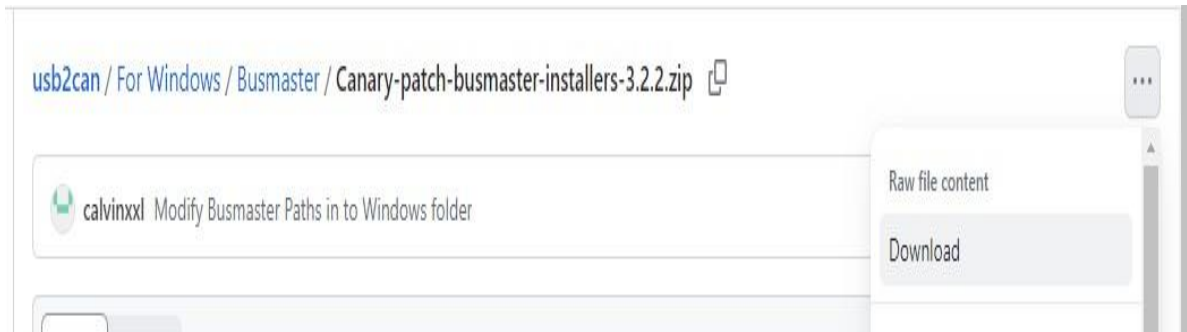
5. open Below link from github [usb2can/Canary-patch-busmaster-installers-3.2.2.zip](https://github.com/usb2can/Canary-patch-busmaster-installers-3.2.2.zip) at master · INNO-MAKER/usb2can · GitHub



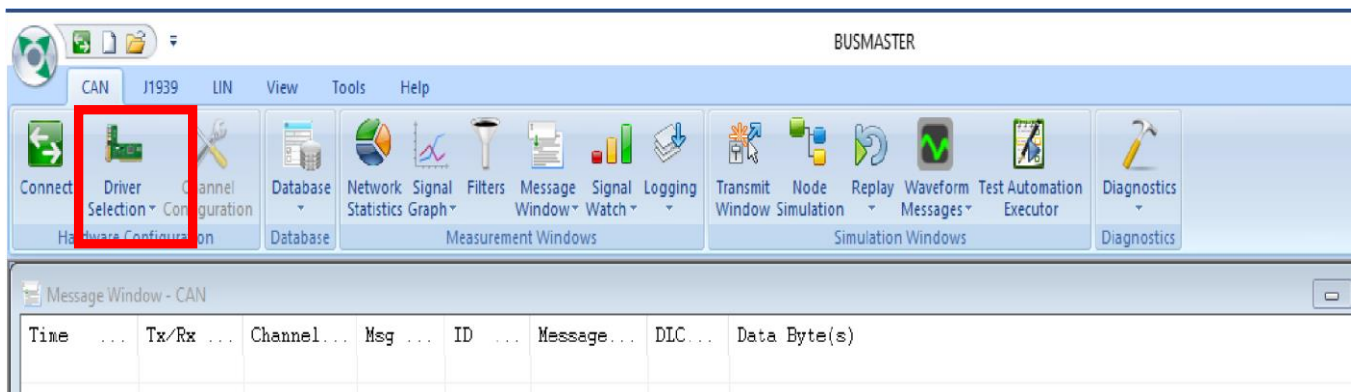
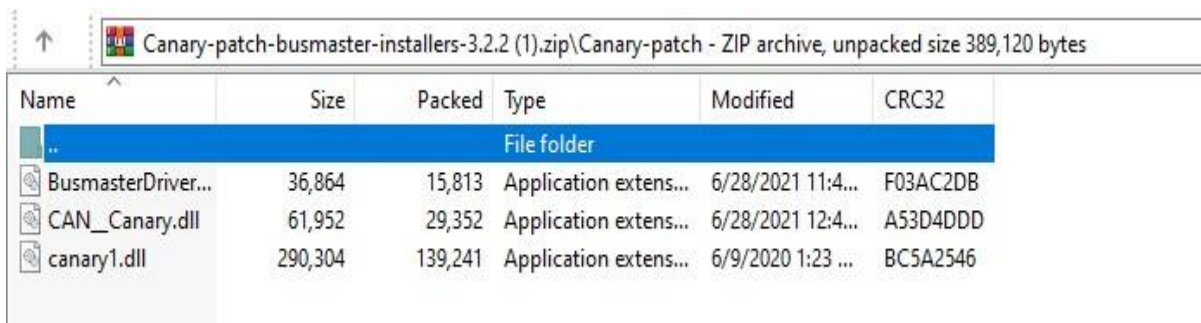
6. Click the( ... )



7. then open the download popup

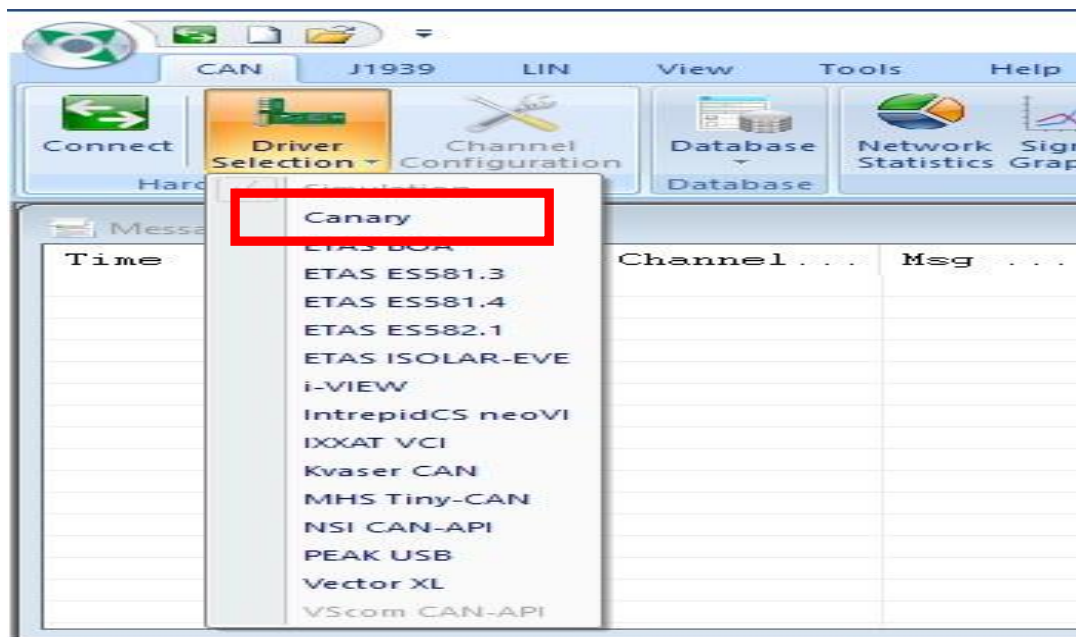


8. Below shown three all files copy and paste to the BUSMASTERv3.2.2 (check the step3 image)



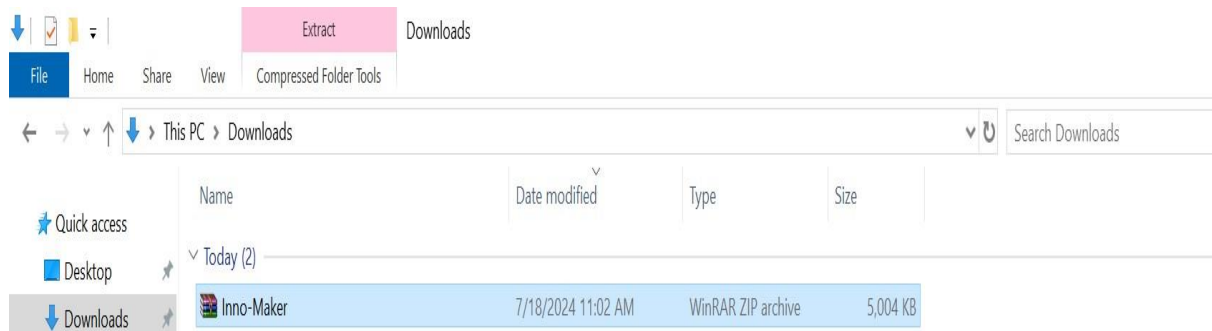
9. Open the bus master application check the driver selection.

10. Check the driver selection Canary is present or not.

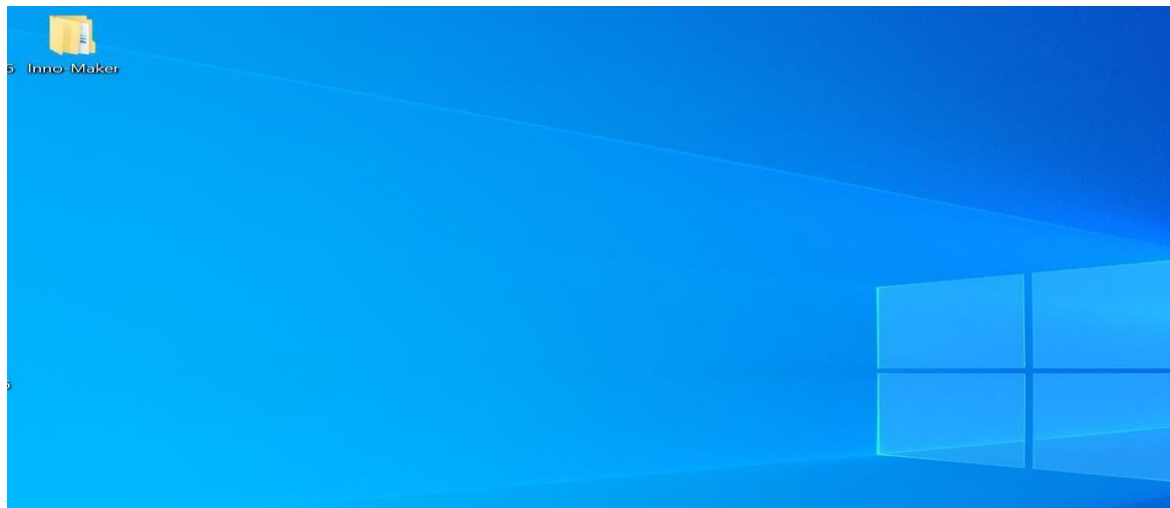


# INNOMAKER DOWNLOAD PROCESS

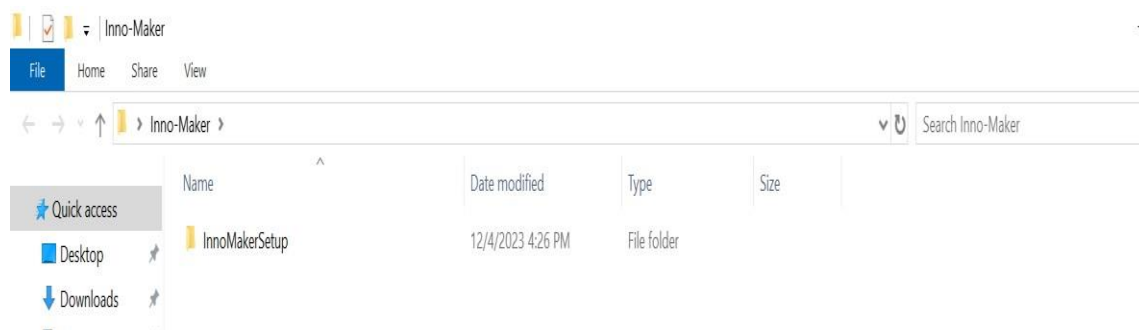
1. Click to download the Innomaker software.



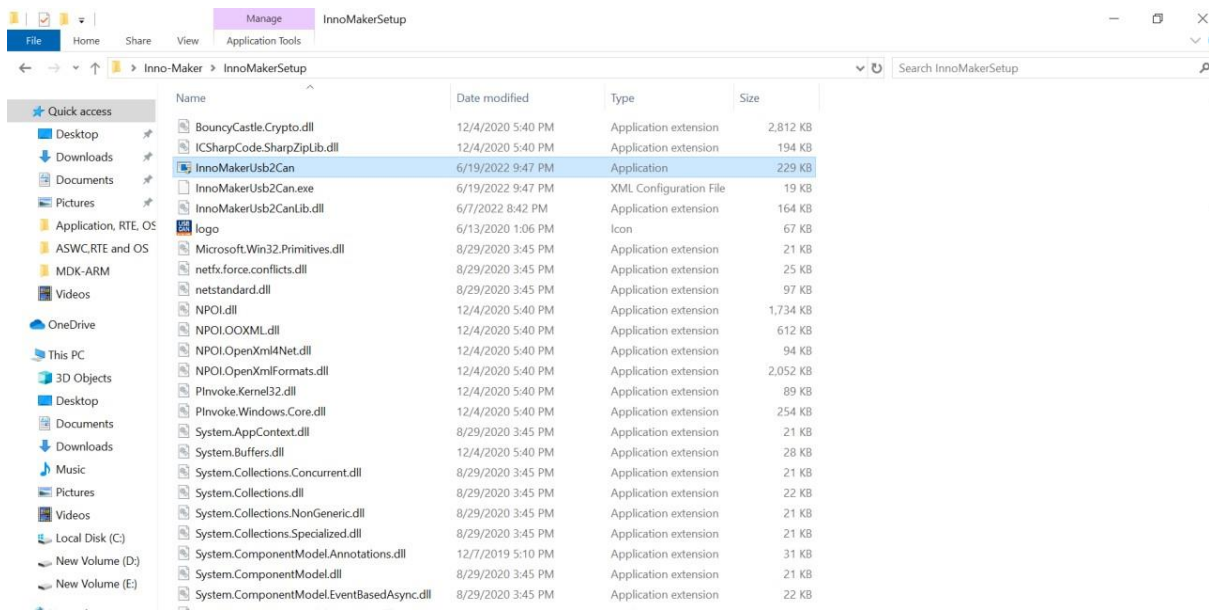
2. Extract the Innomaker files into the C:\ folder.



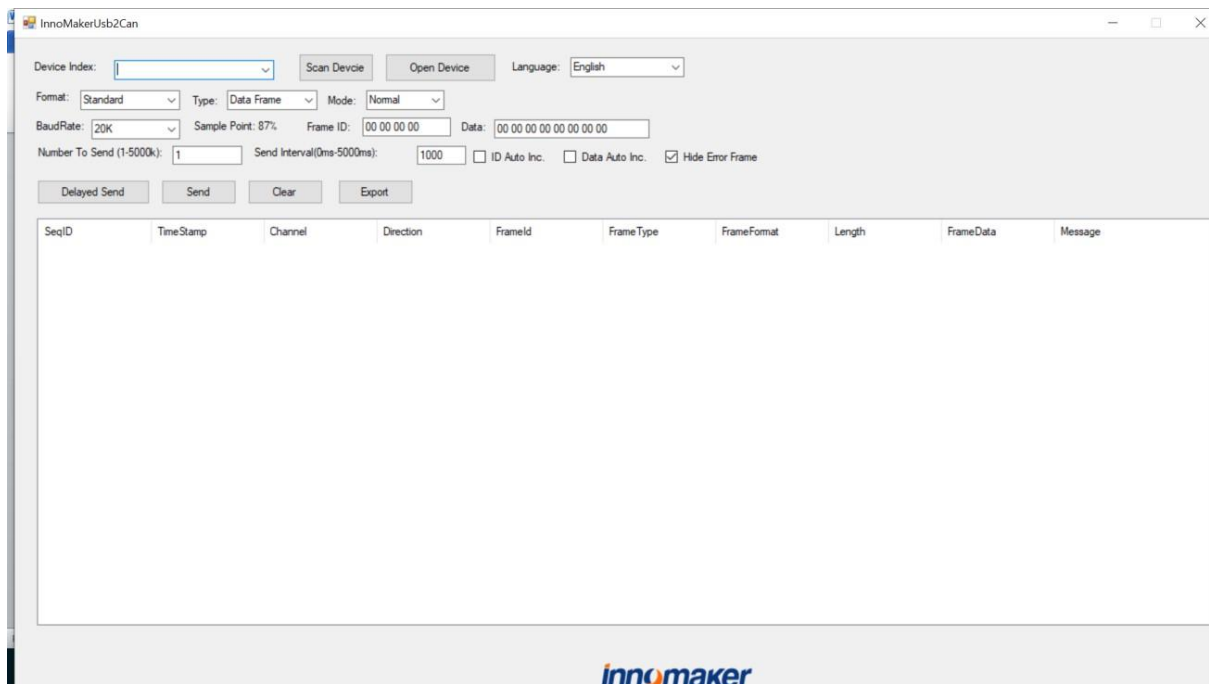
3. Double-click to open the folder.



4. Double-click the Innomaker USB2 CAN application and click 'Yes'.



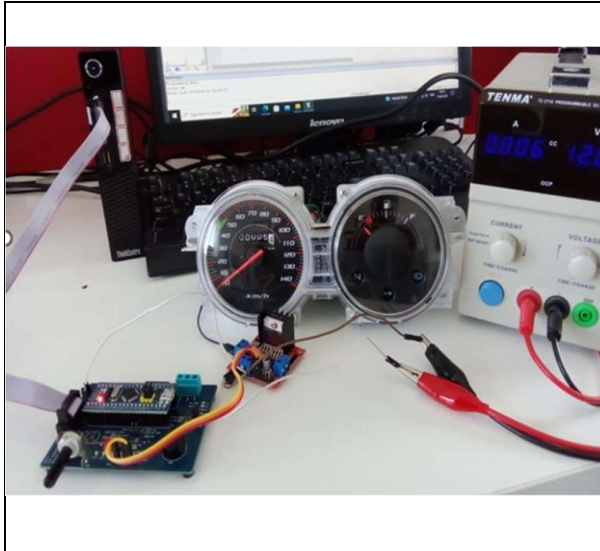
5. You will see this page. That's all.



6. Connect the Innomaker and scan the device. Based on your use case, choose Standard or Extended frame format, set the baud rate and frame ID. Click 'Open Device' to initiate and 'Close Device' to terminate. You can then transmit or receive.

# DEMONSTRATE KITS CREATED USING ANCIT OPEN ECU

## ANCIT Open ECU Demonstrator KIT to Control Head Unit

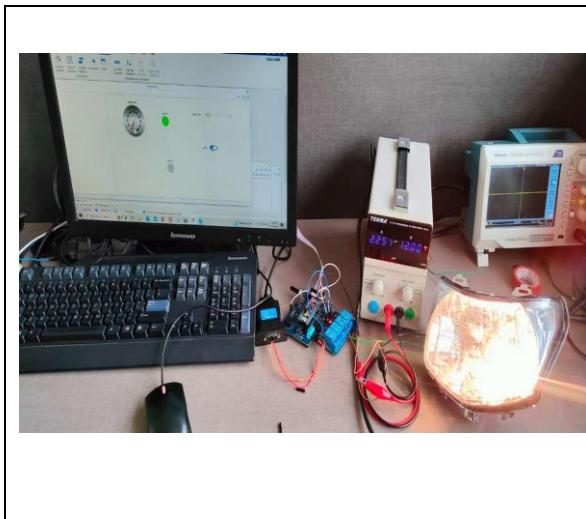


**Following are the features of the board.**

- TS Master [ECU Simulation & Panel Design]
- ANCIT Open ECU [ECU Under Test]
- TENMA Programmable Power Supply
- 2Wheeler Head Unit with Speed & Fuel Gauge
- Motor & Motor Controller for updating Speedometer

<https://ancitconsulting.com/openecu.html>

## ANCIT Open ECU Demonstrator KIT to Turn on Headlight



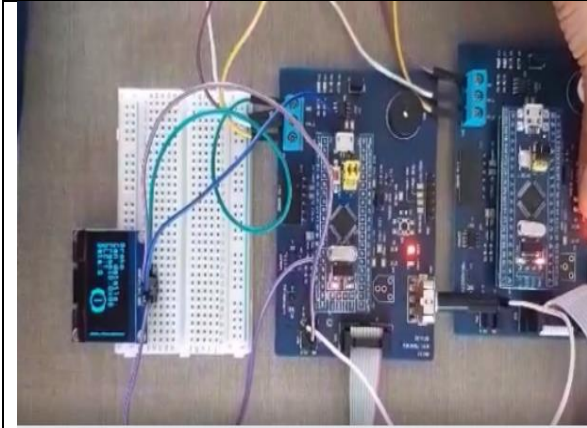
**Following are the features of the board.**

- TSMaster [ECU Simulation & Panel Design]
- ANCIT Open ECU [ECU Under Test]
- TENMA Programmable Power Supply
- 2Wheeler Headlight & Relay

<https://ancitconsulting.com/openecu.html>



### ANCIT Open ECU Demonstrator KIT to Cluster Integration

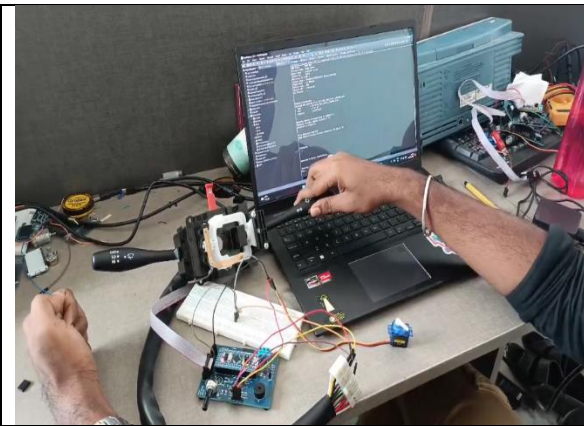


**Following are the features of the board.**

- TS Master [ECU Simulation & Panel Design]
- ANCIT Open ECU [ECU Under Test]
- TENMA Programmable Power Supply
- OLED display
- Motor & Motor Controller for updating Speedometer

<https://ancitconsulting.com/openecu.html>

### ANCIT Open ECU Demonstrator KIT to Wiper Control



**Following are the features of the board.**

- TS Master [ECU Simulation & Panel Design]
- ANCIT Open ECU [ECU Under Test]
- TENMA Programmable Power Supply
- 4Wheeler wiper input switches & servo motor

<https://ancitconsulting.com/openecu.html>

## COURSES OFFERED BY ANCIT

| Sl No | Course Name                                                                                       | Duration in Days | Duration in Hr |
|-------|---------------------------------------------------------------------------------------------------|------------------|----------------|
| 1     | Classic AUTOSAR-4 Weeks Training with AUTOSAR Tool License(EB Tresos)                             | 20               | 160            |
| 2     | Classic AUTOSAR-2 Weeks Training with AUTOSAR Tool License(EB Tresos)                             | 10               | 80             |
| 3     | Classic AUTOSAR-1 Week Training with AUTOSAR Tool License(EB Tresos)                              | 5                | 40             |
| 4     | Classic AUTOSAR- 1 Day Training                                                                   | 1                | 8              |
| 5     | Classic AUTOSAR Training for Business Executives (2 Days)                                         | 2                | 16             |
| 6     | Classic AUTOSAR Diagnostic Stack Training -2 Weeks Training with AUTOSAR Tool License (EB Tresos) | 10               | 80             |
| 7     | Automotive Ethernet and Central Gateway ( 1 Week)                                                 | 5                | 40             |
| 8     | Diagnostics over IP (DoIP) and SOME/IP (2 Days)                                                   | 2                | 16             |
| 9     | CAN and CAN FD (5 Days)                                                                           | 5                | 40             |
| 10    | UDS ( 2 Days)                                                                                     | 2                | 16             |
| 11    | ECU Simulation and Testing (4 Weeks)                                                              | 20               | 160            |
| 12    | Automotive Functional Safety Training (ISO26262)- (4 Days Training + 1 Day Certification)         | 5                | 40             |
| 13    | Cyber Security ISO 21434 training - (3 Days Training + 1 Day Certification)                       | 4                | 32             |
| 14    | Unit Testing using Gtest (1 Day)                                                                  | 1                | 8              |
| 15    | Embedded Training for Freshers (4 Weeks)                                                          | 20               | 160            |
| 16    | Embedded Automotive and AUTOSAR Training ( 8 Weeks)                                               | 40               | 320            |
| 17    | MBD Using MATLAB Training (4 Days)                                                                | 4                | 32             |
| 18    | Embedded Training for Industrial Automation ( 4 Weeks)                                            | 20               | 160            |
| 19    | ISTQB Foundation Level Training (2 Days)                                                          | 2                | 16             |
| 20    | Automotive Protocols (2 Days)                                                                     | 2                | 16             |
| 21    | ISOBUS Training (2 Days)                                                                          | 2                | 16             |
| 22    | Electrical Hardware Training (4 Weeks)                                                            | 20               | 160            |
| 23    | C++ and Basic Linux (10 Days)                                                                     | 10               | 80             |
| 24    | Eclipse Plugin Development Training Using Eclipse IDE, EMF, Xtext                                 | 10               | 80             |



Engineering Tools Provider

Automotive Training Provider



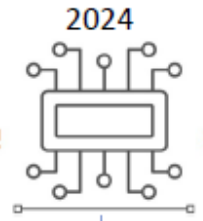
2007



2015



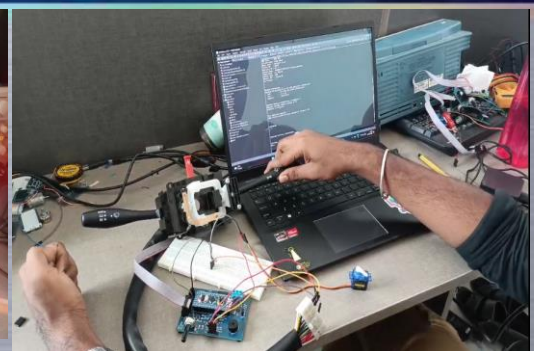
2019



2024

Engineering Services Company

Automotive Tier 1



# ANCIT

**Bangalore Office: SS Residency, 5th Floor, #63, Sri Rama Mandir Road,  
Basavanagudi, Bangalore, Karnataka, India-560004**

**Email id: [info@ancitconsulting.com](mailto:info@ancitconsulting.com) Contact: 9840378602/9789848413**